# 10.PID

## 零.下载matlab

- 祝贺大家通过二面，开始校内赛的准备。和以往任务不同，校内赛会大量涉及电机操控。不同于单片机，编写代码后可以直接读取对应反馈信号；对于电机，我们只能修改 PWM 占空比来间接改变它的转速和转动角度，不能直接指定期望的速度与位置。由于电机存在惯性、摩擦以及外界负载的各种干扰，输出效果无法做到完全可控。这就需要借助控制算法完成闭环调节，从而让电机运转更加精准、平稳。
- 而学习与仿真控制算法必不可少的就是MATLAB，一款强大的数值计算的编程环境，对于解决数学问题极其方便，可以在里面进行复杂的矩阵计算，求解微分方程，绘制图表。可以说仿真控制算法不用MATLAB，相当于学机械建模不用Solidworks。
- 很遗憾，由于tju被制裁了，图书馆买不到正版的MATLAB，所以请参考MATLAB下载安装教程（附安装包）2025最新版（MATLAB R2024b）-CSDN博客下载破解版😋。注意下载的时候要严格按照操作来。安装包记得勾选simulink和simscape（选下），在本教程里面会有用

## 一.算法与控制的基本概念

1. 算法：
    - 经过一段时间的培训，相信大家已经对我们的嵌入式系统有所了解，可以使用GPIO点亮LED,用EXTI去进行按键响应，使用串口来向上位机发送调试信息，使用DMA来自动进行高速数据传输，用TIM来对对电机编码器进行计数估计速度，生成PWM波控制电机转速和舵机。
    - 上面提到的都是基于微控制器片上集成的片上外设而实现的具体功能模块，但是并不是所有的功能都是需要硬件配合，相信你们自己可能也按照自己的想法基于逻辑实现了一些功能，比如根据按下按键的次数来改变电机转速，在使用串口时，根据上位机发送的内容来决定接下来该回应什么内容等。在你解决这些特定问题时，你所采取的这些明确的步骤就可以被泛化到**算法**这一个概念。
    - 算法包含的内容非常广泛，有信息竞赛的排序，查找算法，加密数据的加密算法，对信号进行处理的滤波算法，以及我们本节将会学习的控制算法。
2. 自动控制系统的相关概念：

这里我们以洗澡为例进行说明




- **受控对象** 需要被控制的设备，也即水温
- **执行器** 直接作用于被控对象，使其状态发生改变的装置，他接受控制器的命令，输出“控制作用” ，也即上个例子中的放水按钮
- **传感器** 测量被控对象的输出，并将其转换成易于处理的电信号反馈给控制器。**反馈是自动控制的核心，**也即上个例子中你的皮肤和温度传感器
- **控制器** 系统的大脑，他接受设定值与传感器反馈值的偏差,并按照某种预定的控制算法，进行计算，产生给一个送给执行器的控制信号，也即你和浴室的把手。
- **期望值** 期望系统输出达到的目标值，也即人体泡澡水温度适宜区间
- **扰动** 会对系统输出不利影响的外部因素，也即旋钮的摩擦力

3. 反馈：

就是将系统输出的信号送回给输入端，并与输入信号进行比较得到的结果。分为**正反馈**和**负反馈**

- **负反馈**：与实际差别越大往回拉，一般我们用的都是这种。
- **正反馈**：与实际差别越大越会放大偏差。

4. 开环与闭环控制
   - **开环** 无反馈，仅凭借设定期望值产生控制作用
     - 显然这对洗澡是一场灾难，对于左拧一度炼钢，右拧一度挂霜的把手来说，这种不负责任的行为会对你的皮肤造成重创。
   - **闭环** 就是我们提到的自动控制系统
     - 可以预见的是，只要温度传感器精度足够和响应速度足够的情况下，你可以不断按照自己的感受调整把手，直到稳定在人的舒适温度附近。
   - 而对于我们接下来的应用，即小车上的话，一个直观的例子就是小车轮电机的转速，当我们仅施加了某占空比的PWM波进行调速(PWM与电机转速的严格数学模型未知)后，我们实际上并不知道实际的速度与我们设定的是否一致，由此我们需要引入编码器进行轮电机速度的反馈来形成闭环控制系统，来更精确的控制速度。
5. 就我们将会使用的控制算法而言，还有些重要的特征需要注意，它们可以评价一个算法代码的好坏

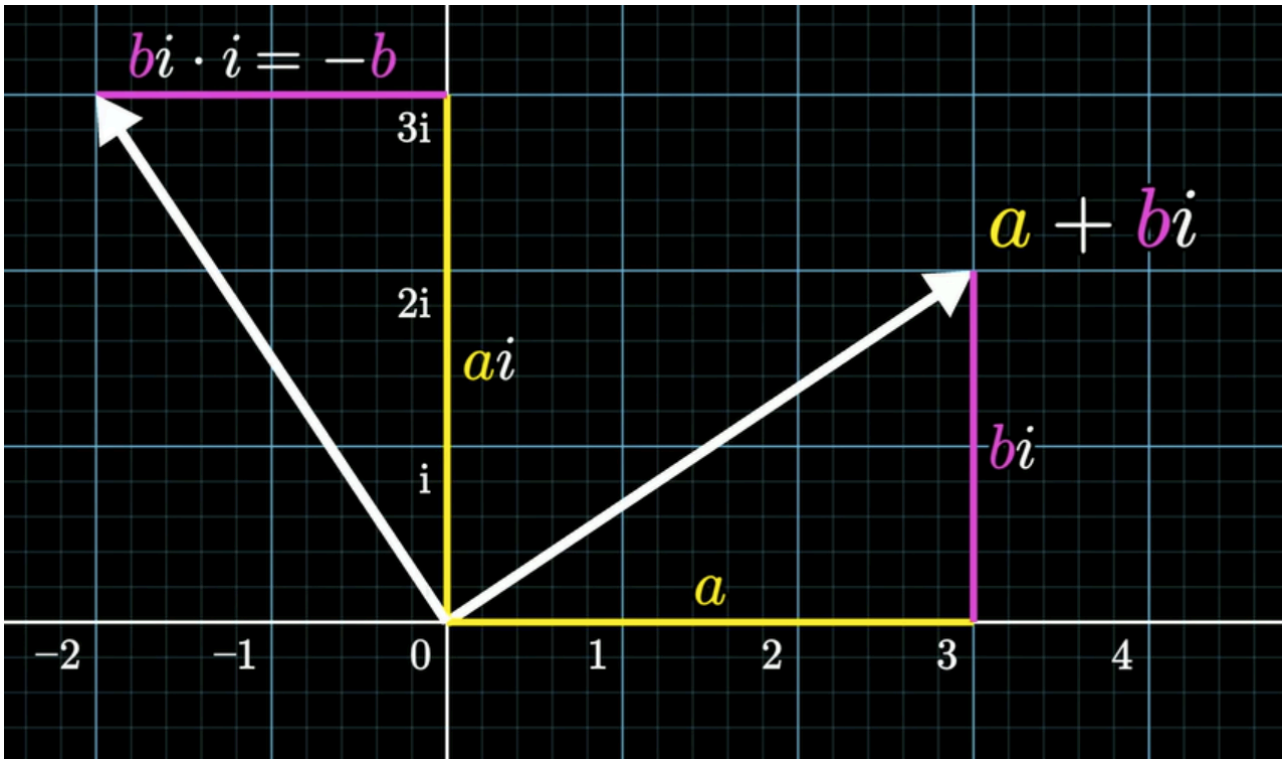
- **效果** 包括响应的速度(对于变化的参考信号，系统跟随的速度)，稳态误差(系统最终是否能够收敛到设定值)，稳定性(系统在控制过程中是否会发生震荡而不是平稳的收敛到最终值)
- **明确性** 算法的步骤，变量必须被明确描述，影响你代码的可读性和移植性
- **时间效率** 算法运行一次所占用的时间，算法时间效率过低会导致控制频率降低，分辨率降低，稳定性降低，干扰同时进行的其他任务
- **空间效率** 算法会占用多少内存空间资源 嵌入式设备空间有限，必须斟酌所占用的空间

# 二.数学基础

> 下面的数学基础看不懂其实不太影响pid的学习，但可以让你对控制有一个基本的了解。所以下文选看，最后一段有省流版本。如有由于表述不严谨等问题导致的晦涩难懂敬请谅解。

## 1.欧拉公式

- 我们高中就学过，实数轴和虚数轴可以构成一个复平面，复数可以在复平面上表示。如果说负数是180度的翻转（想象表示2和-2的矢量在实数轴的关系），那么我们可以把乘上虚数理解成90度的转动。




参考这张图可以便于理解在i(a+bi)=-b+ai为什么就等同于复平面上90度的转动

- 我们知道，路程的求导是速度，我们有 $\frac{\mathrm{d}e^{st}}{\mathrm{d}t} = se^{st}$ ，如果把 $e^{st}$ 当做路程，那么 $se^{st}$ 就是速度。比如对于路程和时间的关系有 $x(t) = e^{t}$ 的质点，其速度在数值上等于路程。那么就意味着在质点运动到x的时候，它的速度在数值上也是x。

- 我们可以把这个道理延拓到复平面上，如果 $s=i$，那么就有 $\frac{\mathrm{d}e^{it}}{\mathrm{d}t}=ie^{it}$，而我们知道复平面上的只会改变复平面矢量的方向，而不改变大小，那么这就意味着这个质点做速度在数值上与路程一样，但是方向垂直的运动。什么运动速度和位移的矢量始终垂直？**圆周运动！**
- 同时我们知道 $t=0$ 时路程为1，那么速度也始终为1。所以 $e^{it}$ 的本质就是复平面上在半径为1的圆上做速度为1的圆周运动。那么 $t=\pi$ 时，刚好转到-1的位置，也就有 $e^{i\pi}=-1$。结合单位圆的知识，我们就得到了大名鼎鼎的欧拉公式 $e^{i\theta}=\cos\theta+i\sin\theta$（当然也可以使用泰勒展开直接证明，但我认为这么理解更形象）,它揭示了三角函数和指数函数的虚数次方之间的联系。

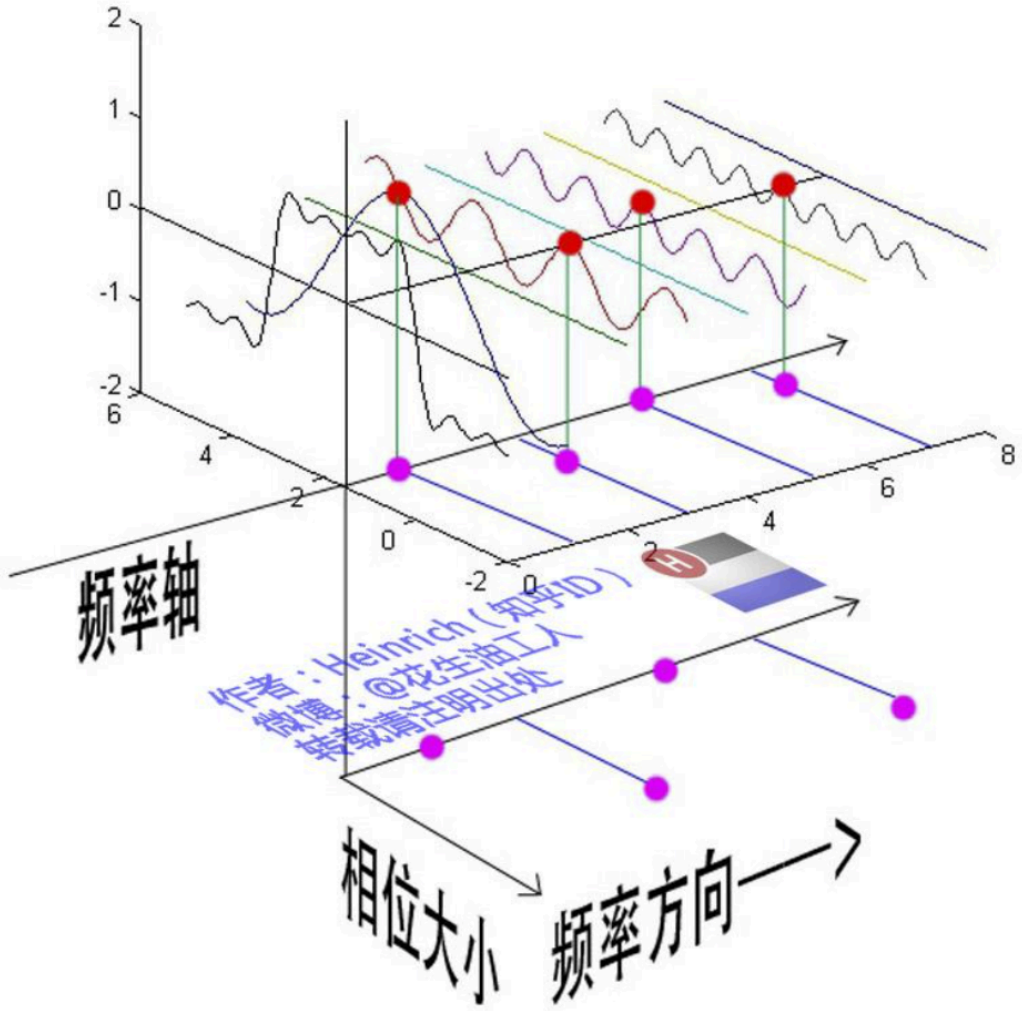
## 2.拉普拉斯变换

- 你可能会问这很有趣，但是有什么用呢？事实上这意味着对于 $e^s$ 这一指数函数（s是复数），我们可以把它拆分为实部的指数函数和虚部的三角函数。而现实中的很多运动都可以由这两个函数描述。三角函数意味着运动的周期性，指数函数，随着a<0，=0，>0，可以分别描述衰减，恒定，放大的三种趋势。两种函数的结合可以描述弹簧振子，电机，电路等各种各样的物体，所以找到一种运动的方程是由什么样的指数函数和三角函数组成，对研究它的控制非常重要，而将指数函数延拓到复平面的欧拉公式为我们提供了非常重要的桥梁。
- 但是事实上，我们接触到的运动往往都是时间的微分方程的形式，而微分方程的形式对于计算结果是很困难的。举个例子



图有点丑，后续会换个好看点的

- 这是一个经典的弹簧振子模型，竖直摆放的弹簧受到与速度成正比的空气阻力，弹簧弹力和恒定重力的作用，那么由牛顿第二定律，设弹簧运动与时间的关系函数为 $x(t)$ ，那么就有 $mx''(t)+px'(t)+kx(t)=g(t)$ （这里设g(t)是恒为10N的重力，p是空气阻力系数），我们把这种与时间相关的方程称为时域上的方程。显然当微分方程阶数越来越大（最高项求导次数越大），方程求解就越困难，我们也就越难以得到这个运动的频率，运动趋势等相关性质。这时我们就会引入一种控制领域非常重要的操作——拉普拉斯变换，公式为 $F(s)=\mathcal{L}\{f(t)\}=\int_0^{+\infty}f(t)e^{-st}\mathrm{d}t$ 。这个变换**把原本时域上的方程转换到复频域上与复数s相关的方程**。
- 拉普拉斯变换的式子中，从0到无穷积分，我们可以找到使得 $F(s)$ 区域无穷大的s值，称为极点。式子内部乘上的复指数 $e^{-st}$ 中，**实部的含义是找原函数中的指数函数，虚部的含义是找原函数中的三角函数**。当寻找到匹配的三角函数和指数函数后，积分结果会发散。因此一个已知解析式的函数经过拉普拉斯变换，它的极点就暗含了原本函数可以由什么样的三角函数和指数函数组成。同样的我们知道拉普拉斯变换后的式子，也可以反推原本的形式与具体的系数，这个我们在本章第五节进行说明。
  - 当拉普拉斯变换中 $s$ 的实部变成0的时候，就是大名鼎鼎的傅里叶变换 $F(\omega)=\mathcal{F}\{f(t)\}=\int_{-\infty}^{+\infty}f(t)\,e^{-i\omega t}\mathrm{d}t$ 。由于实部等于0，所以傅里叶变换关心的更纯粹——它只关心原函数由什么样的三角函数构成。我们这里以此为例子看看它是怎么寻找到组成该运动的三角函数的。
  - 傅里叶变换的精髓就是三角函数的正交性 $\begin{cases}\int_{-\pi}^{\pi}\sin mt\ \sin nt\,\mathrm{d}t=\begin{cases}0, & m\neq n\\ \pi, & m=n\neq 0\end{cases}\\ \int_{-\pi}^{\pi}\cos mt\ \cos nt\,\mathrm{d}t=\begin{cases}0, & m\neq n\\ 2\pi, & m=n=0\\ \pi, & m=n\neq 0\end{cases}\\ \int_{-\pi}^{\pi}\sin mt\ \cos nt\,\mathrm{d}t=0\end{cases}$
  - 感兴趣的同学可以用积化和差自行证明。由正交性，所以只有 $\omega$ 经过原函数中包含的三角函数的时候，其积分结果才不为0。因此当一个 $\omega$ 代入后积分不为0，就意味着原函中包含着该频率的三角函数。
  - 这里给出一张图，大家可以借图片理解一个普通的函数是怎么被拆分成多个三角函数的，这些三角函数在频率上又是怎么表示的。




## 3.传递函数

- 还是回到例子里的方程 $mx''(t)+px'(t)+kx(t)=g(t)$，它的左边的 $x(t)$ 是预期的输出路程，右边的 $g(t)$ 是预期的输入力，两边同时做拉普拉斯变换（这里涉及到拉普拉斯变换的核心公式之一，当初始条件 $f(0)=0$ 时，有 $\mathcal{L}(f'(t))=sF(s)$，可以通过分部积分法证明，此处略），可以得到 $X(s)(ms^2+ps+k)=G(s)=\frac{10}{s}$ .进一步化简，有 $X(s)=\frac{10}{s\left(ms^2+ps+k\right)}$，这个式子被称为 $x(t)$ 的输出响应。注意s=0的极点来自输入，而 $ms^2+ps+k=0$ 的两个根的极点来自系统运动本身，我们关注的是系统自己的性质（因为输出可以改，系统本身改不了），所以关注两个根所表示的极点。我们将 $H(s)=\frac{X(s)}{G(s)}=\frac{1}{ms^2+ps+k}$ 称为这个运动的输入和输出之间的传递函数（传递函数还要求零初始条件，也就是输出及其各阶导数都是0，即 $x(0)=x'(0)=x''(0)=0$。当然绝大部分情况这个都是满足的）。这里当分母等于0时，$H(s)$ 趋近于无穷，也就找到了这个系统本身的极点。通过极点的坐标得到组成该运动的三角函数和指数函数，进而就可以分析其性质。(实部a<0，则系统本身运动的振荡分量会逐渐收敛到0，实部a>0，则运动会发散，虚部b越大，则弹簧阻尼振荡频率越高，结合这两个函数性质想想为什么)
    - 虽然我们主要关注系统本身的极点，但是这并不意味着输入的极点没有意义。事实上系统最终的总输出的性质是由系统本身和输入共同决定的。稳态是**外力输入和系统内部力达到平衡的结果。**

    - 我们还是拿这个例子说明。极点s=0对应原函数是常函数，再结合系统是衰减的，因此我们可以知道，最后的输出是逐渐衰减到一个固定值的 $x(t)$ ，这个结果也是符合现实的。
    - 一般而言，系统决定 “怎么去”，输入决定 “最终去哪”
- 因此从传递函数的方面，我们可以得到拉普拉斯变换另一个重要的意义：它把微分方程中难以求解的关系转化到了复频域中简单的乘法关系，也即 $Y(s)=H(s)X(s)$ ，或者说输出象函数等于输入象函数乘上传递函数。由此当微分方程难以求解时，我们可以进行拉普拉斯变换，知道系统和输入的极点，我们就可以推测出输出的基本形式。（拉普拉斯变换后的复变函数就叫象函数）

## 4.一阶惯性系统

- 一阶，指的是方程里面最多出现到一阶导数。惯性，指的是对系统的外部影响并不会立刻生效，而是逐渐发生影响。
- 还是拿滑块做例子，如果此时滑块在液体中滑动，受到液体的作用力 $g(t)$ 。由于没有弹簧了，我们输出项从 $x(t)$ 改成 $v(t)$ ,我们就可以列出来 $mv'(t)+pv(t)=g(t)$ ，这里的m就是惯性，当液体密度突变，阻力 $g(t)$ 增大时，速度不能突变，而是会受到惯性的影响，想要改变速度 $v$ ，就必须有加速度项 $v'$ 。我们不可能瞬间把速度拉到期望值，我们必须先产生加速度慢慢爬升。而p就是阻尼，会阻止速度的产生（理解一下这两个的区别）。这样的系统就是一个典型的一阶惯性系统。（注意一阶惯性系统必须包含阻尼项 $pv(t)$ ，不然就成积分了，比如 $s(t)=\int v(t)dt$ ,路程也不能跳变，但是它是积分，不是一阶惯性系统）。
- 而下文中很快会出现的 $K_p$ 的计算就是典型的非惯性系统。 $U(t)=K_p*e(t)$ ，可以看出来，当 $e(t)$ 出现跳变， $U(t)$ 会马上跳变，不存在惯性的说法
- 两者区别也可以从传递函数中看出来。滑块系统的传递函数 $H_1(s)=\dfrac{1}{ms+p}$ ，而路程与速度的传递函数是 $H_2(s)=K_p$ （相信大家的积分功底😊）

## 5.拉普拉斯逆变换

- 正如我们之前说的，知道拉普拉斯变换的结果就可以反推原本的系数。我们得到了传递函数，再通过逆拉普拉斯变换就可以得到原函数的组成。我们这里介绍一种运用较广的版本——部分分式分解。
- 首先我们需要知道两个基本变换 $\mathcal{L}^{-1}\left\{\dfrac{1}{s+a}\right\}=e^{-at}$ ， $\mathcal{L}^{-1}\left\{\dfrac{1}{(s+a)^2}\right\}=te^{-at}$ 。基于此，我们可以把拉普拉斯变换后的 $Y(s)$ 和 $X(s)$ 的关系拆成多个分式的和。
- 我们以 $\dfrac{4s+5}{s^2+5s+6}$ 为例。它可以拆分为 $\dfrac{c_1}{s+2}+\dfrac{c_2}{s+3}$ ，以求 $c_1$ 为例，两边同乘 $s+2$ ，有 $F(s)(s+2)=\dfrac{4s+5}{s+3}=c_1+\dfrac{c_2}{s+3}(s+2)$ ，注意这里左边的 $s+2$ 可以代入 $F(s)$ 里面吸收掉，之后代入 $s=-2$ ，就有 $c_1=\dfrac{4\cdot(-2)+5}{-2+3}=-3$ 。同理 $c_2=7$

- 由此可以知道 $F(s)=\frac{-3}{s+2}+\frac{7}{s+3}$ ，两边做拉普拉斯逆变换，得到

$$f(t)=\mathcal{L}^{-1}\left\{\frac{-3}{s+2}+\frac{7}{s+3}\right\}=-3e^{-2t}+7e^{-3t}$$

### 省流版本

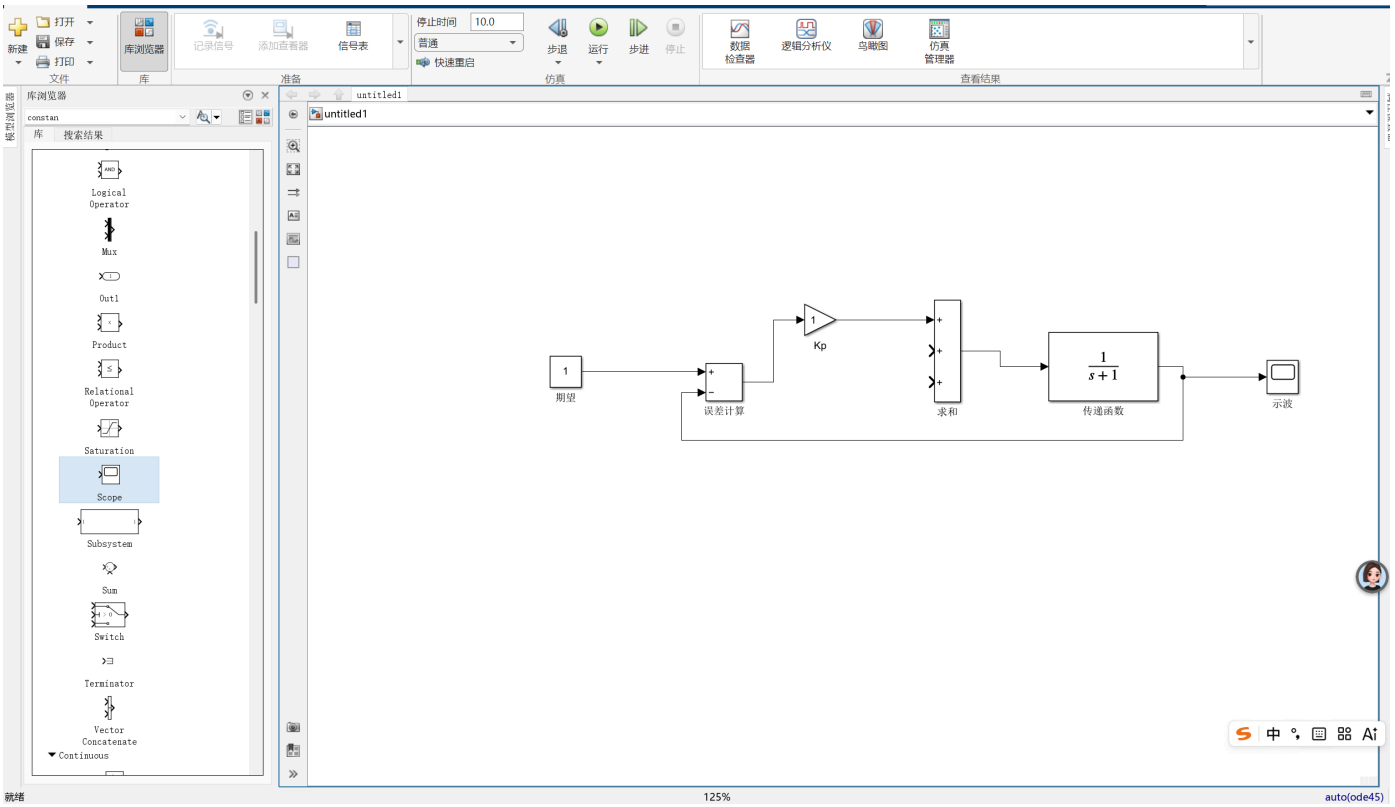
- 在后续的pid仿真中，我们输入的传递函数，就暗示了这个系统经由输入得到的输出的敛散性和频率等关键性质。
- 一个有阻尼项，不能消除稳态误差的就是一阶惯性系统。能消除静态误差的就是非惯性系统

## 三.抽象流PID介绍

- 看完上面的公式，可能大家有点头昏了，所以在这一部分，我会减少公式，用相对通俗的方法解释PID。同时结合matlab进行说明。
- 顾名思义，PID由三个部分组成，比例项P（proportional），积分项I（integral），微分项D（derivative），它是一种非常符合直觉，应用也非常广泛的控制算法。

### 1.比例项P

- 还是以洗澡举例子，人体洗澡的舒适温度大概是38摄氏度，但是可能刚出的冷水就10度，那么这个时候我们就会加大力度拧把手，试图快速的靠近理想温度。同样的温度差不多了力就会变小
- 用公式表示就是 $U(t)=K_p*e(t)$ ，e就是误差(error)
- 但是我们可以预计，只有一个P很多情况（比如一阶惯性系统）是无法到达预期温度的，因为系统的扰动会对受控对象产生作用，为了抵消误差，Kp的作用会让系统最终处于一个距离预期目标有距离的平衡点，也就是产生了稳态误差。
- 我们可以通过matlab的simulink功能进行展示。
  - 打开matlab，在上方菜单栏选择simulink，随后选择simulink里面的空白模型。
  - 进入simulink后，在上方菜单栏选择库浏览器，搭建如图所示的仿真。



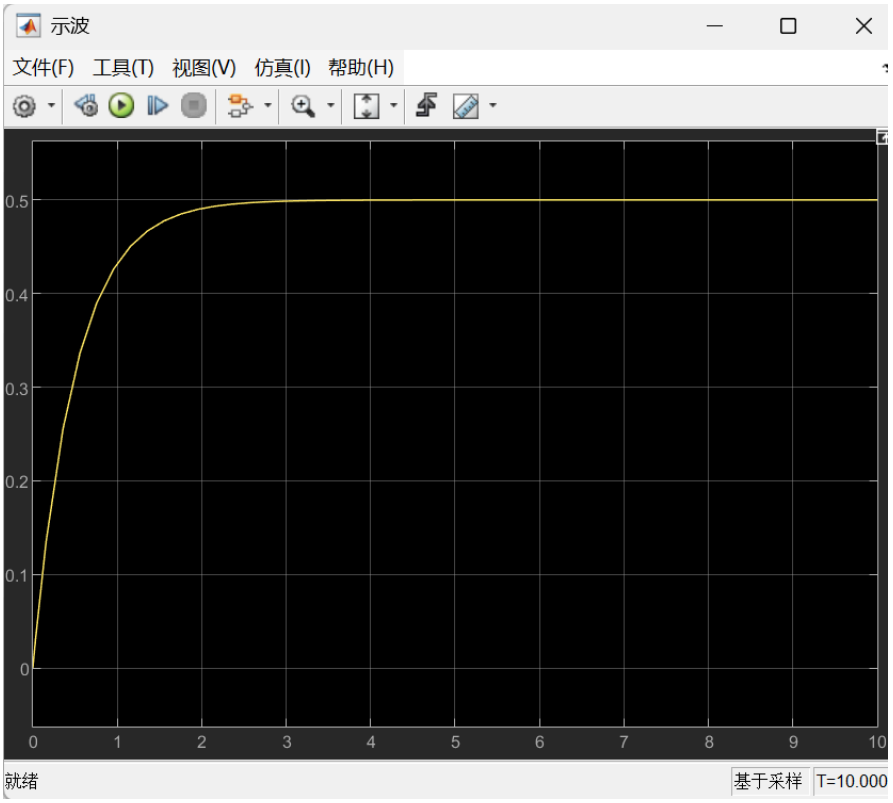

结合中文注释和Kp的计算流程，理解这个仿真

- 按照从左到右，模块分别是`1.simulink->commonly used blocks->constant`，`2.simulink->math operations->add`，`3.simulink->commonly used blocks->gain`，`4.simulink->math operations->add`，`5.simulink->continuous->transfer fcn`，`6.simulink->commonly used blocks->scope`。
  - 双击模块可以进行编辑和修改数值，比如传递函数这里可以在分母系数输入[1 1 1]得到 $s^2+s+1$ 的分母。



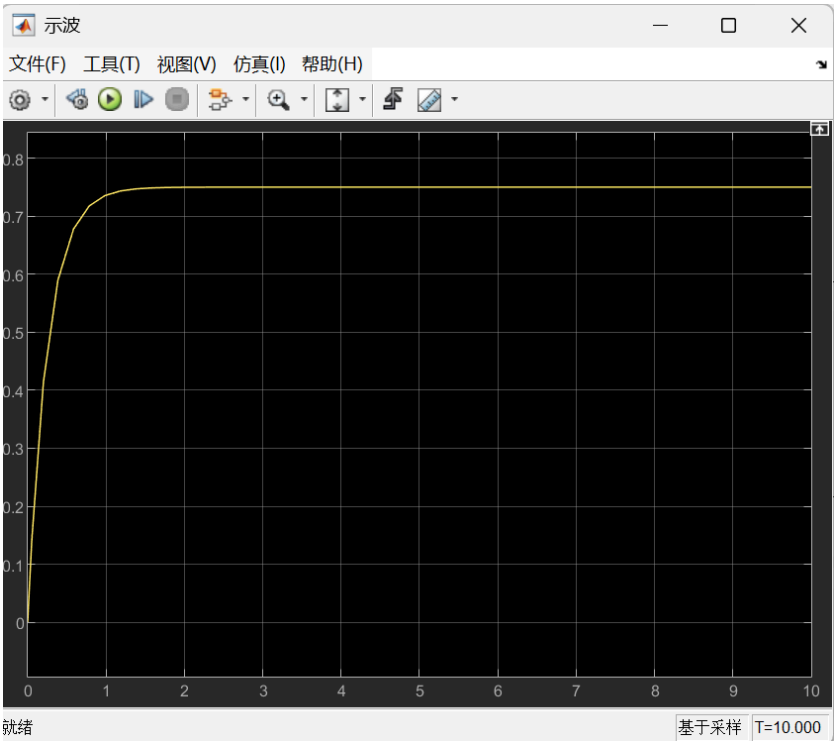

- 这里传递函数模块的计算逻辑是通过传递函数反推出原本的微分方程，将PID计算结果代入输入的部分，得到输出。

- 点击菜单栏的运行，仿真，启动😋！

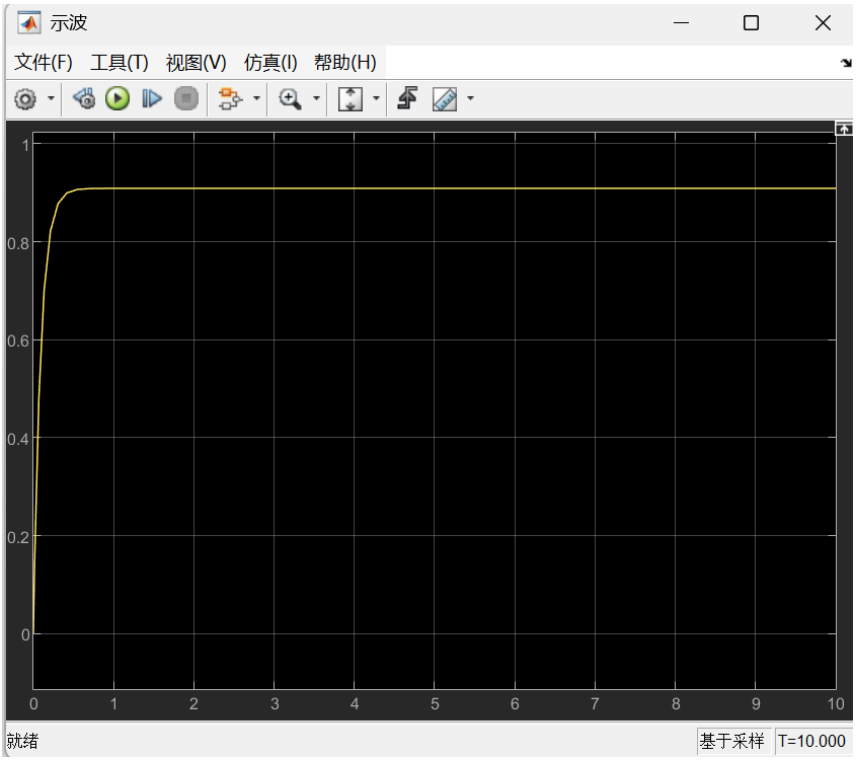



Kp=1

- 可以看到最终运行结果收敛于0.5附近，并没到达1，也就是产生了稳态误差。




kp=3

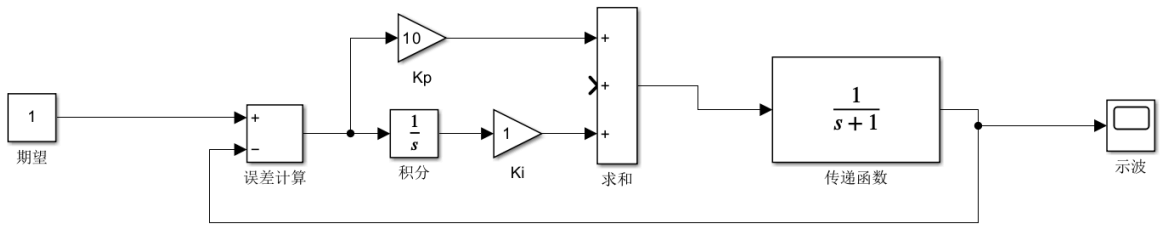



kp=10

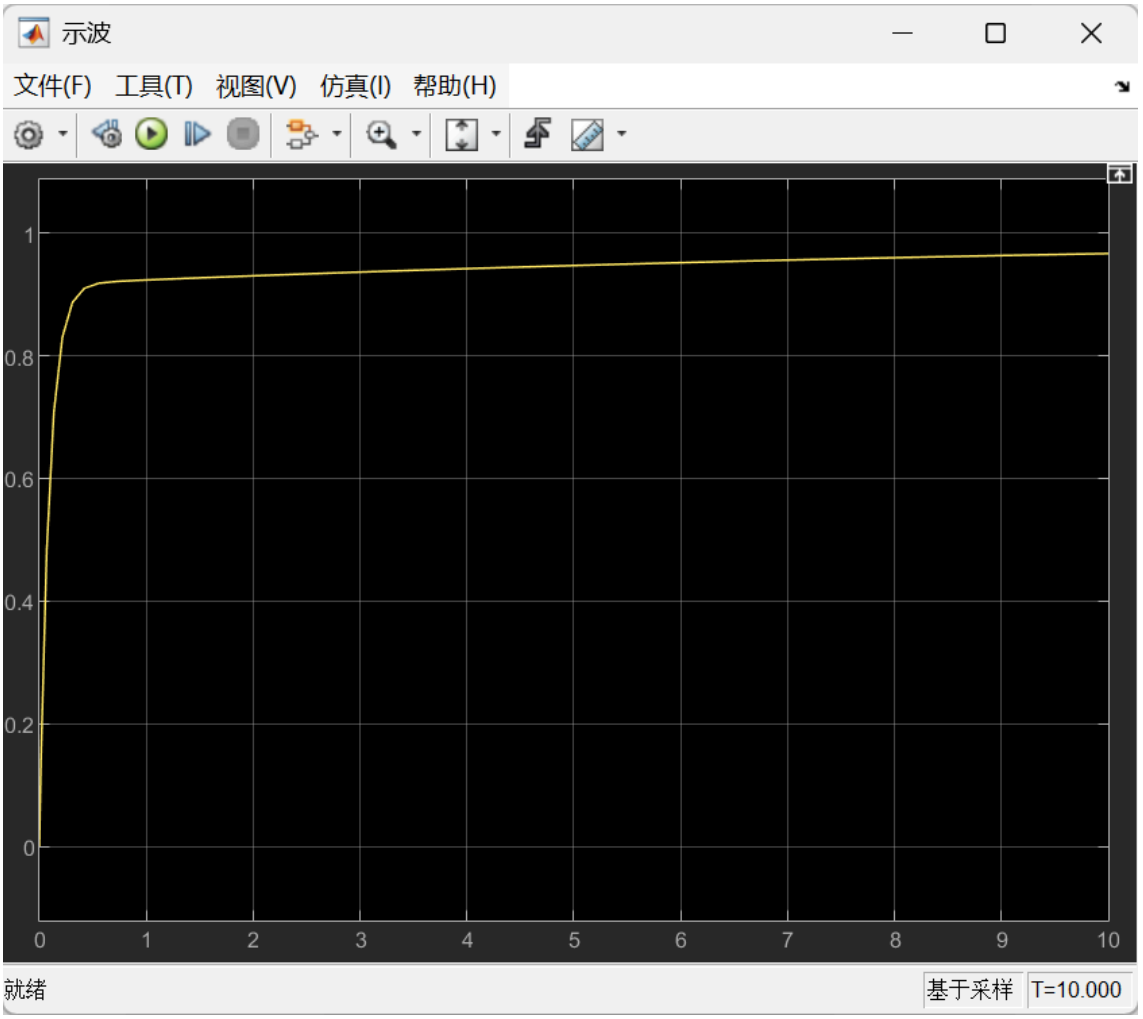
- $K_p$ 的增大确实可以拉近与期望值的距离，但始终无法到达，存在稳态误差。因此我们需要引入更劲的东西，也就是积分项 $K_I$ 🤓。

## 2.积分项I

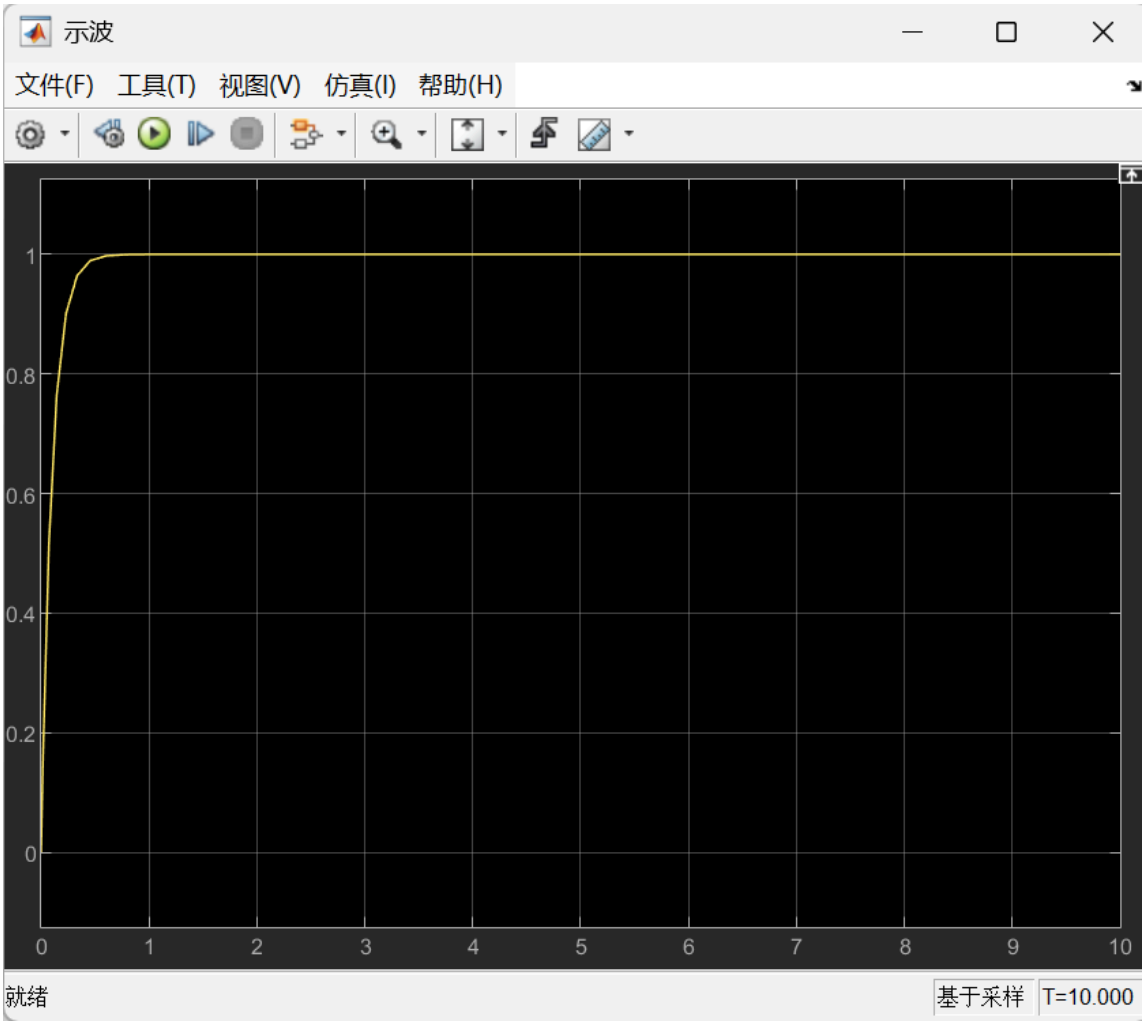
- 为了打破稳态误差，一种合理的想法是引入一个积分，通过不断积累误差使得数值不断变大，打破稳态误差。就像洗澡中一直洗着冷水澡必然焦躁不安，逐渐红温，于是当红温到达阈值之后便会用力一拧，打破这个稳态的平衡，也就起到了拉近目标的作用。
- 用公式表示就是 $U(t) = K_i * \int e(t)dt$
- 积分项确实强而有力，但是积分项 $K_i$ 过大很容易引发一个问题——超调，也就是当前值暴涨到远超期望值而停不下来。事实上**超调在我们的调试中是很经常发生的，只是超调量大与小的区别**。在受控制量逼近于参考值的过程中，误差一直在减小，但始终存在，且正负始终保持一致(仅讨论在第一次受控制量相等于参考量之前)，所以在比例和积分的协同作用下，受控制量会在某一瞬间与参考值相等，此时比例控制的输出为0，但积分控制由于误差始终朝着一个方向积累，其输出一定不为0（且不小）。故系统并不会达到稳态，而会朝着原来受控制量增大的方向继续增加，使误差向另一个方向增大。就像洗澡调不出理想温度红温了，用力拧一下就直接超出了理想温度，直接被狠狠烫到。
- 尽管如此，只要参数调整得当，PI控制在相当多的应用场景中已经足够了。尤其是在一阶惯性系统中。
- 现在我们在simulink里面加入I项控制



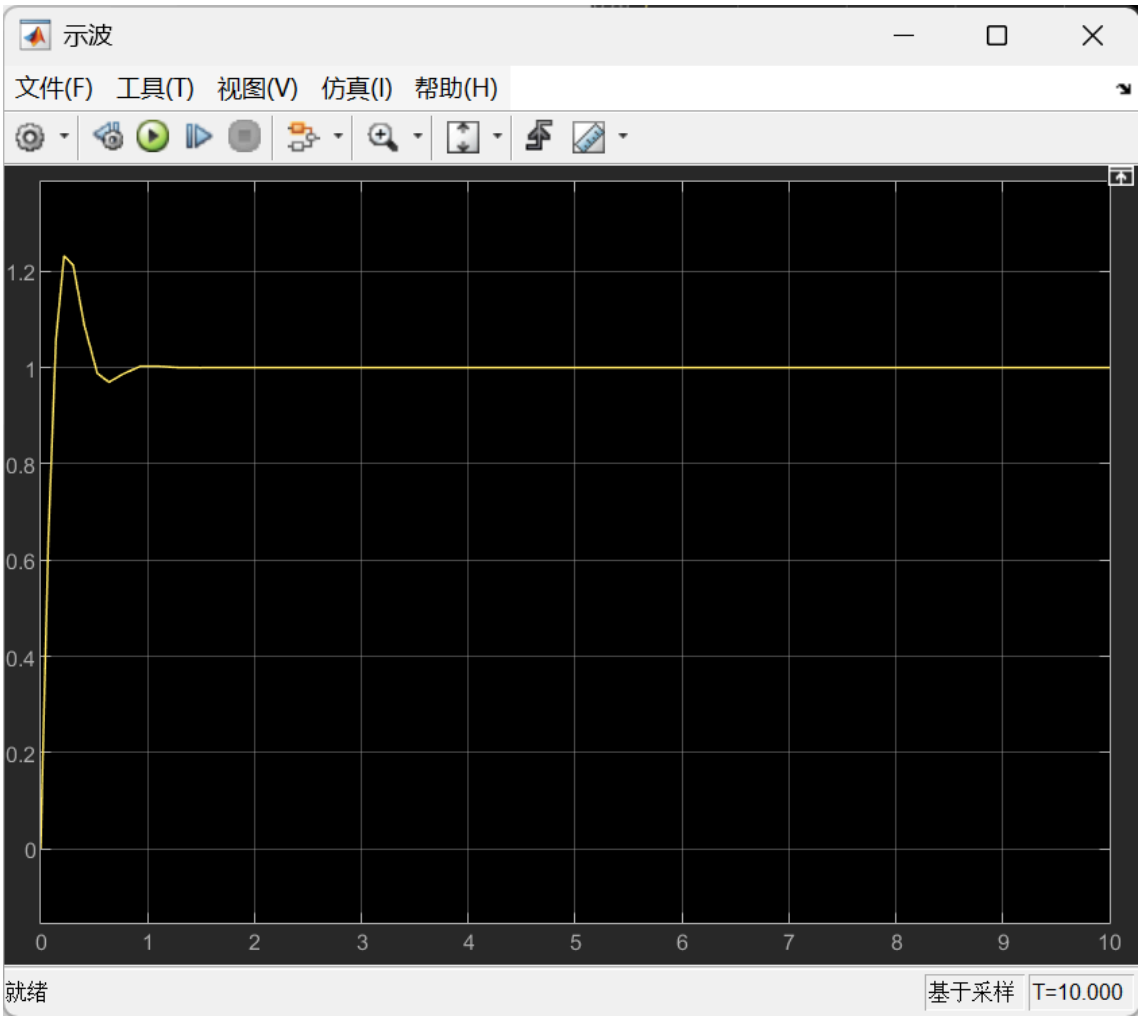

- 这里面多出来的模块是`simulink->commonly used blocks->Integrator`，另一个是之前用过的gain
  - （补充说明）为什么积分是用 $\frac{1}{s}$ 表示呢？因为在时域上做积分时，有 $y(t)=\int x(t)dt$，两边做拉普拉斯变换（还记得 $\mathcal{L}(f'(t))=sF(s)$ 吗，它意味着在零初始条件下，一个函数先求导再拉普拉斯变换等于先拉普拉斯变换再求导，这里就用上了），则 $Y(s)=sX(s)$，所以复频域上积分的表示形式就是 $\frac{1}{s}$。
- 仿真，启动😋！




Kp=10，Ki=1


示波
文件(F) 工具(T) 视图(V) 仿真(I) 帮助(H)
1
0.8
0.6
0.4
0.2
0
0 1 2 3 4 5 6 7 8 9 10
就绪
基于采样
T=10.000

Kp=10，Ki=10



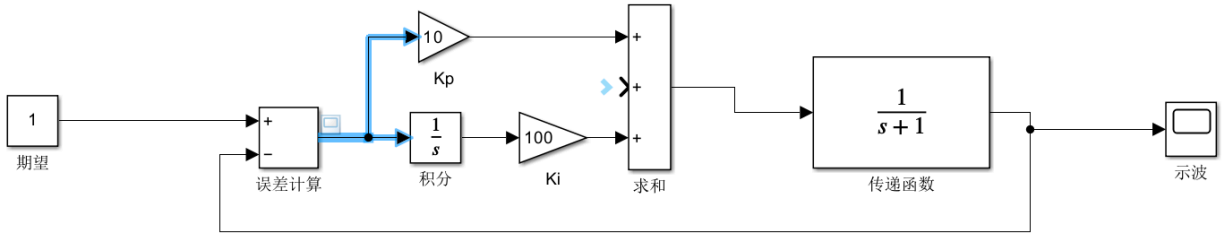

Kp=10，Ki=100

- 可以看到，Ki等于10的时候，我们的系统基本到了比较良好的状态。而Ki暴增到100的时候，出现了比较明显的超调现象。

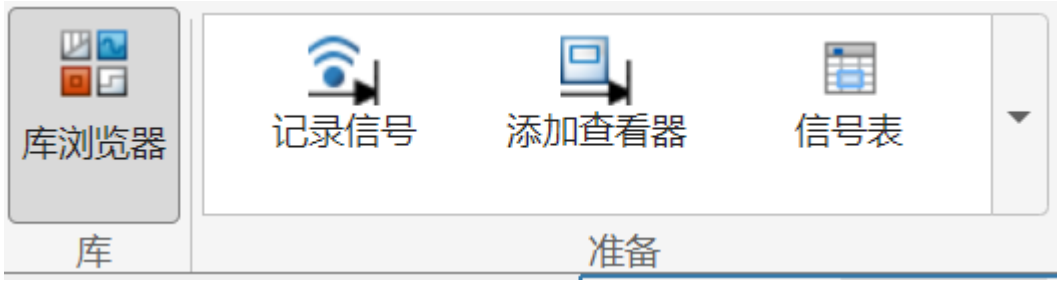
- 在简单的控制中PI已经足够了，但是很多时候的控制其实是个黑盒，比如控制电机，电机的阻力矩，过热，磨损，负载扰动都会对控制有影响，并且这些影响会随时间变化，难以整定成具体的参数。面对复杂的情况，PI控制就显得捉襟见肘，很容易超调。这个时候我们就会引入超调的严父——微分项

## 3.微分项D

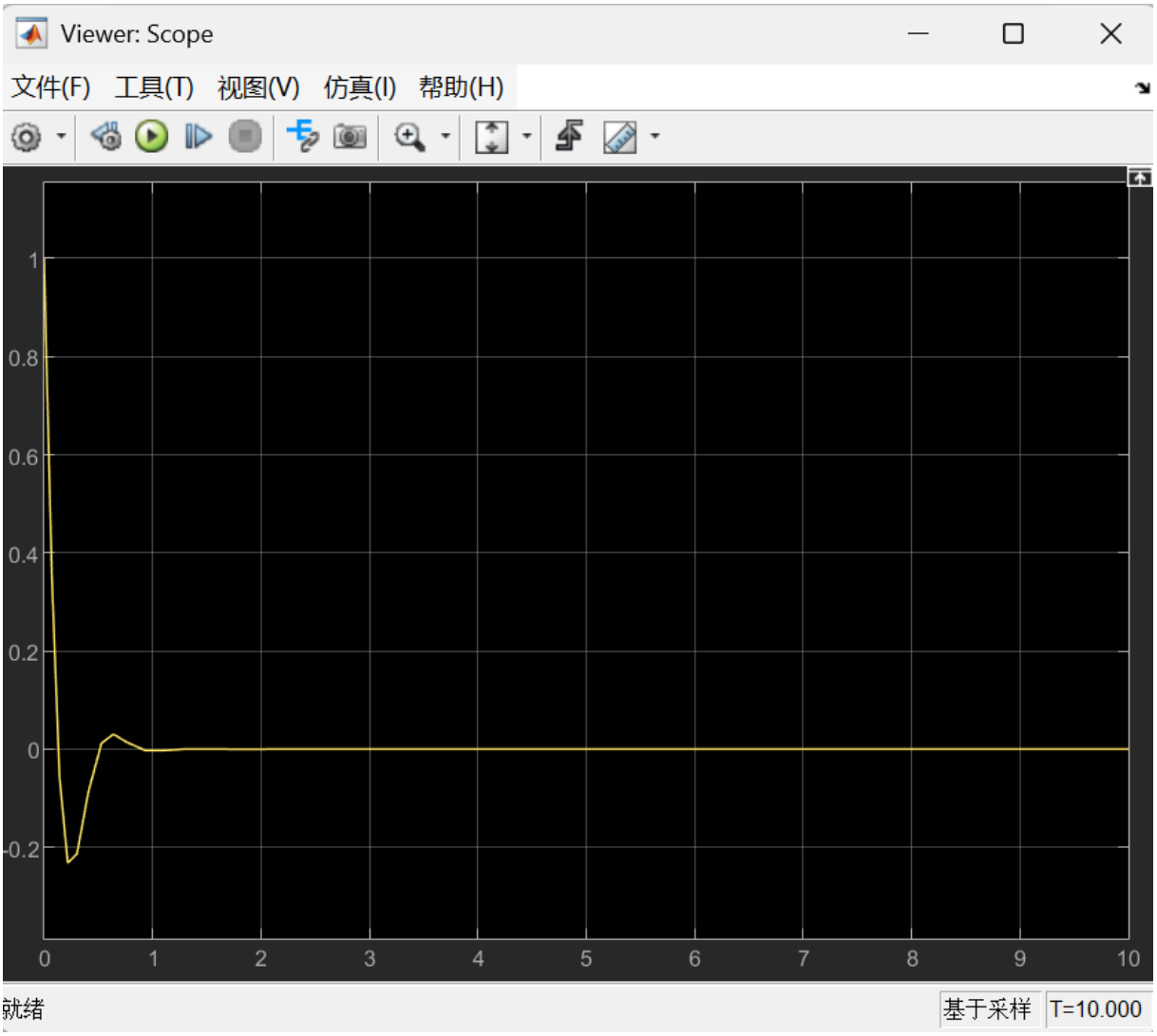
- 为什么会出现超调，因为一开始期望和现实差距太大，积分项快速积累,导致到达期望值的时候积分项积累的很大，快速越过期望值。那么在一开始快速逼近目标的过程中，有什么东西是较大的负数，可以抵消积分项呢。没错，误差e的导数 $\frac{de}{dt}$ 。
  - 我们可以通过simulink看看这个想法合不合理




- 如图，选中这根线，在上方菜单栏选择添加查看器，再选择scope

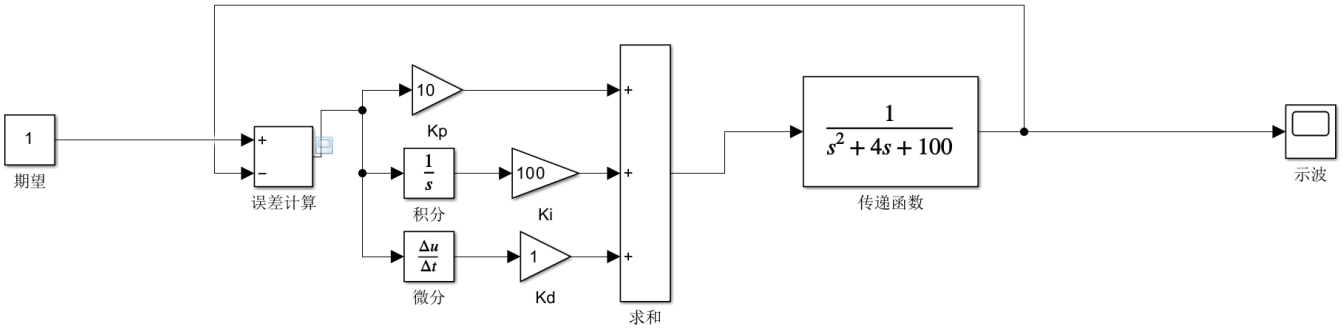



- 再次点击仿真，点击这根线上出现的小scope，可以得到如下误差e随时间变化的图像 $e(t)$

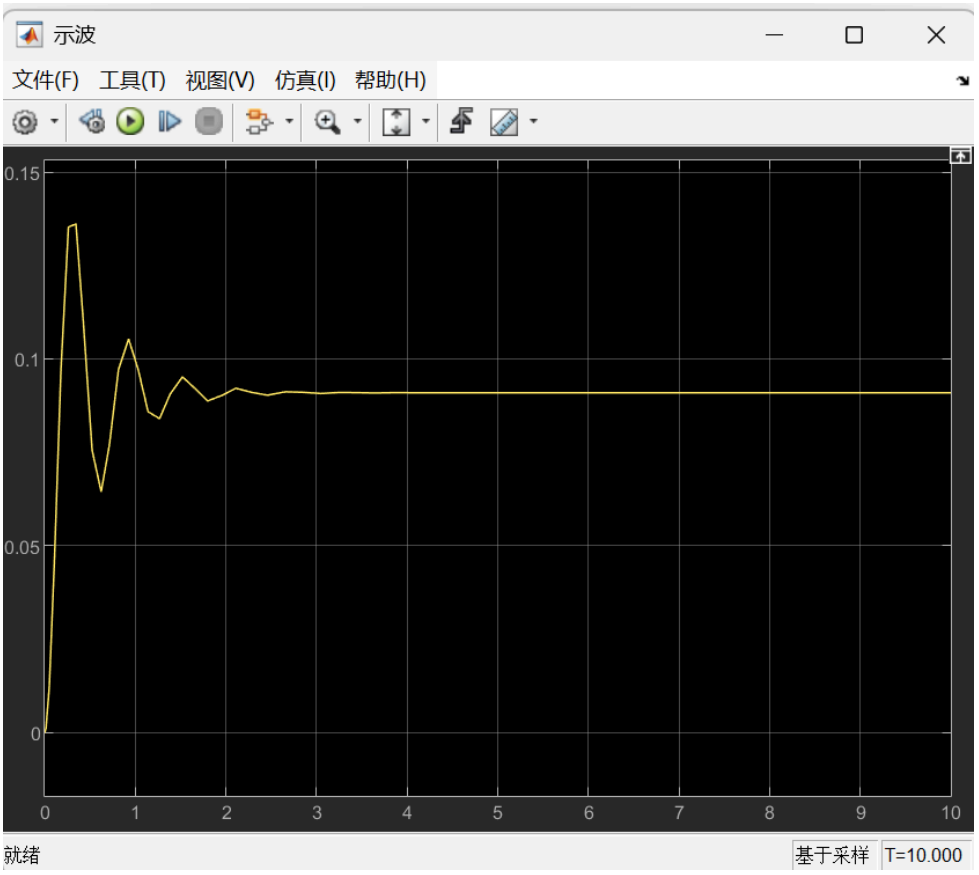


    - amazing!在超调的过程中，图像的斜率始终是较大的负数，所以猜想是合理的，我们可以通过引入微分项，压制住超调

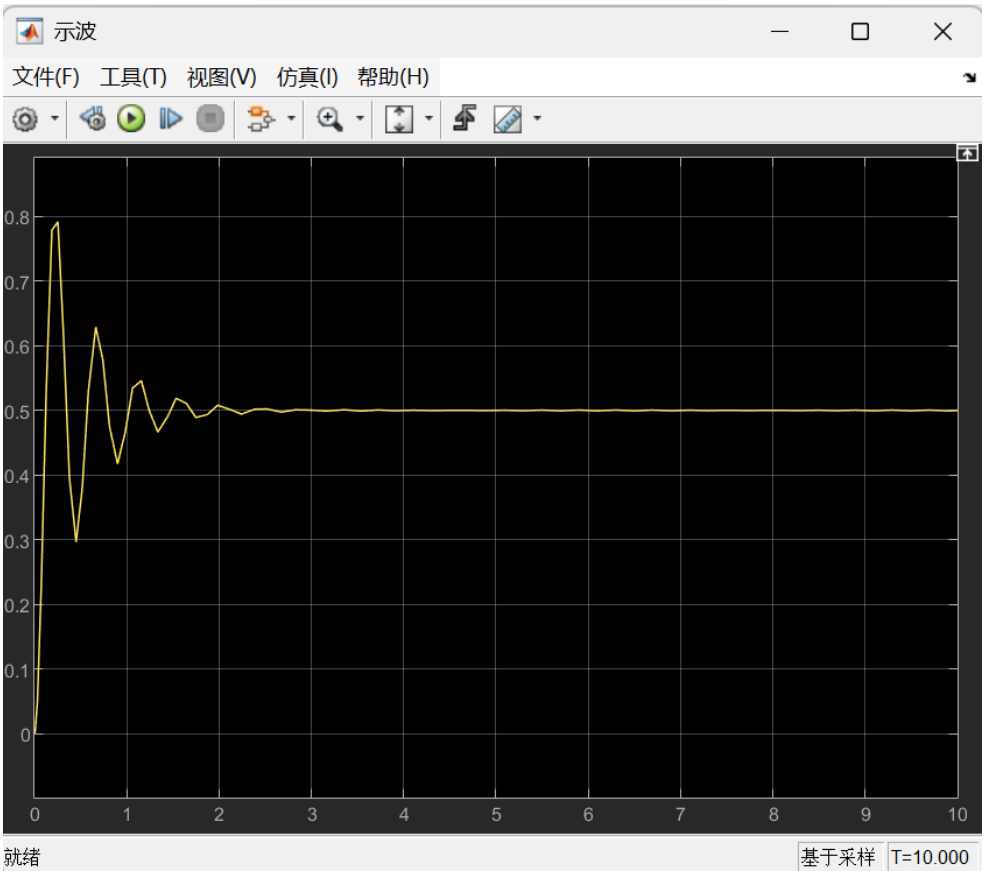
- 用公式表示，就是 $U(t) = K_d * \frac{de(t)}{dt}$
- 在一些较为简单的非惯性系统中，比如计算里程和车辆转向，可以使用PD控制。但就像上面说的，绝大部分现实情况是一个黑盒，所以我们一般还是采用都是PID控制。加入微量的Ki来平衡。在参数整定的过程中，基本的调参方法是参数归零，先调P后I后D,将P调至能够快速响应并且无震荡，加大I使终值收敛，并加快静态和动态响应速度，允许有一定超调，在调节D使超调量减小。
- 当然听着简单，但是这只是理论上的，PID的调参本身是一个考验经验且比较费时间的事情。需要不停地调参-测试-调参，很多时候让人头大的不是算法代码，而是好的参数调不出来😭
- 接下来我们再次打开simulink进行仿真，这次我们选用较为复杂的传递函数 $\frac{1}{s^2+4s+100}$



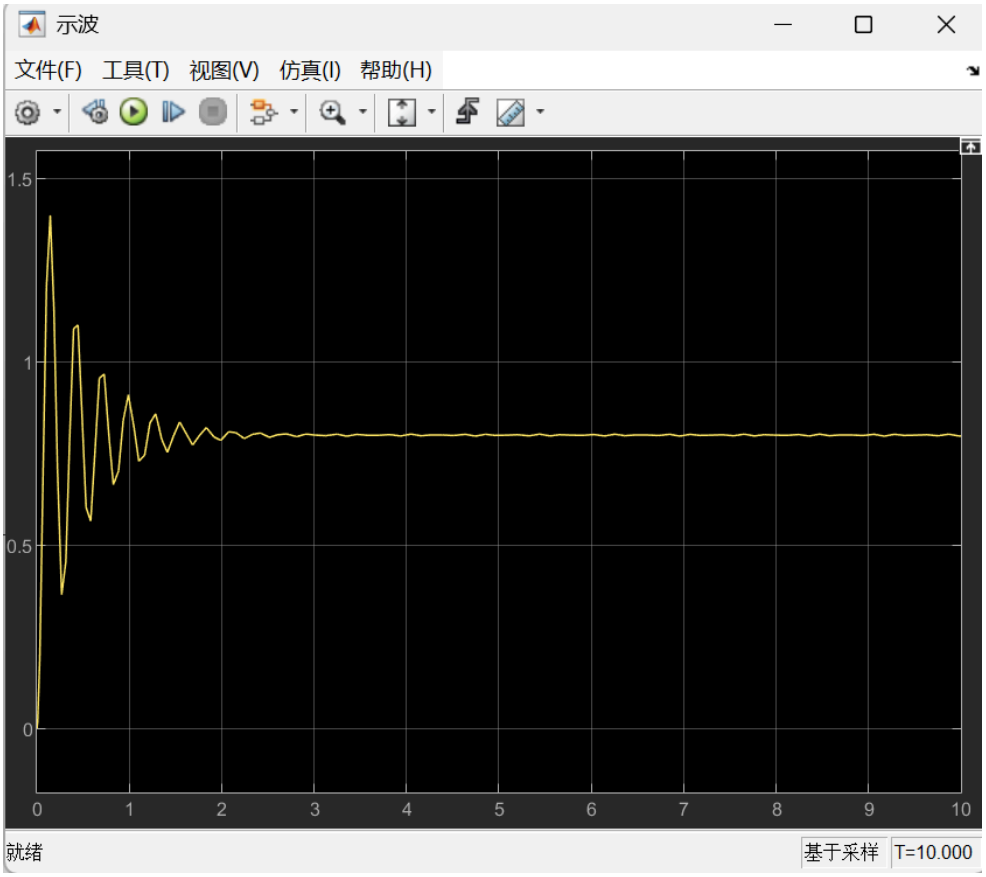

    - 这里新增的模块是 `simulink->continuous->derivative` 以及老朋友gain。



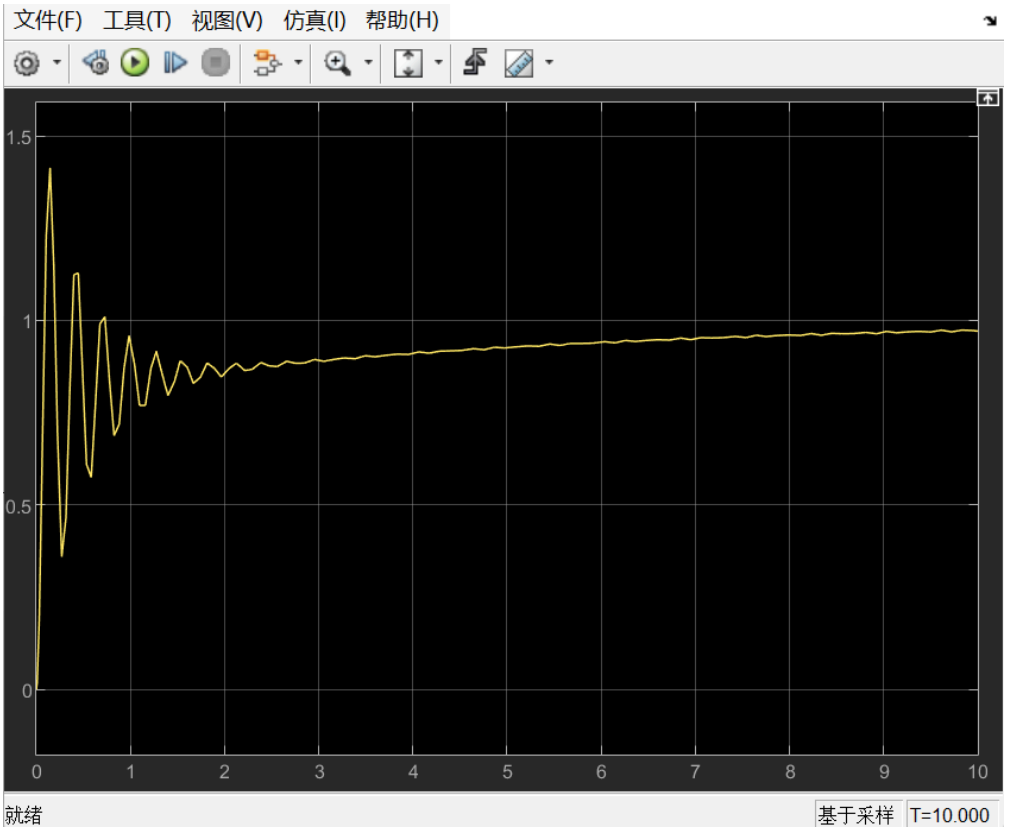

Kp=10，Ki=0，Kd=0



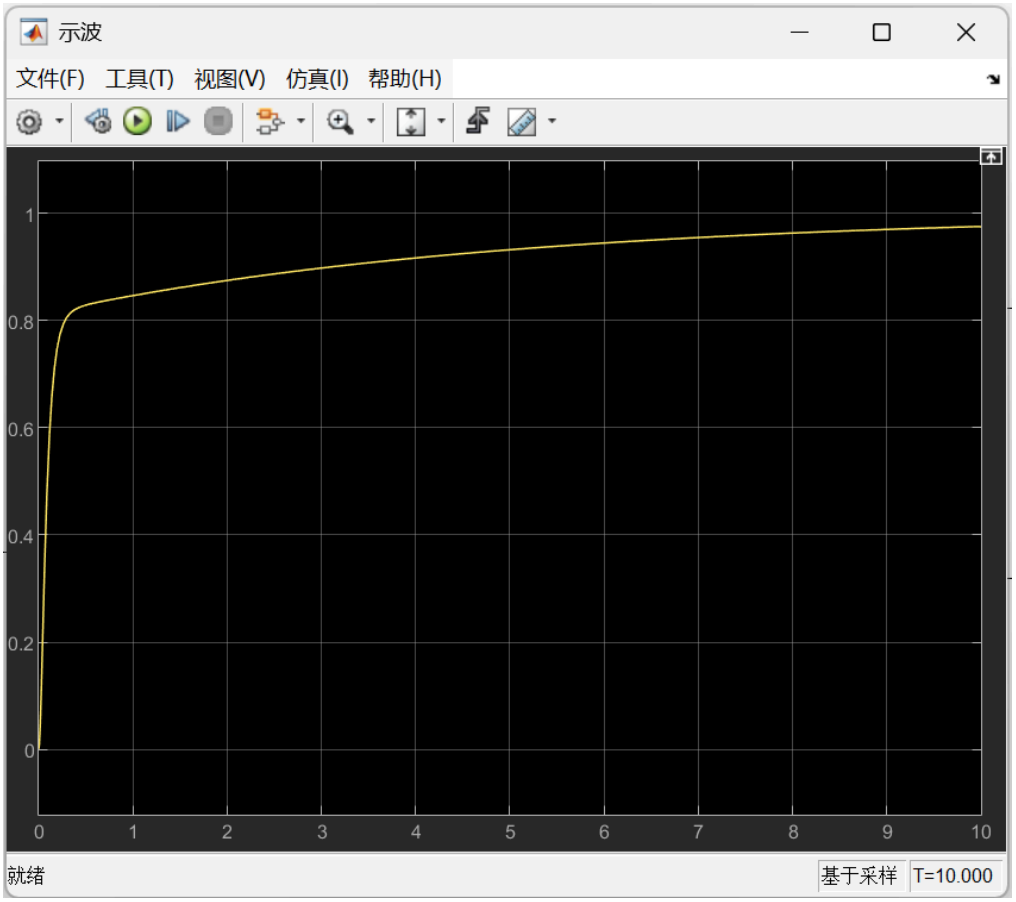

Kp=100，Ki=0，Kd=0



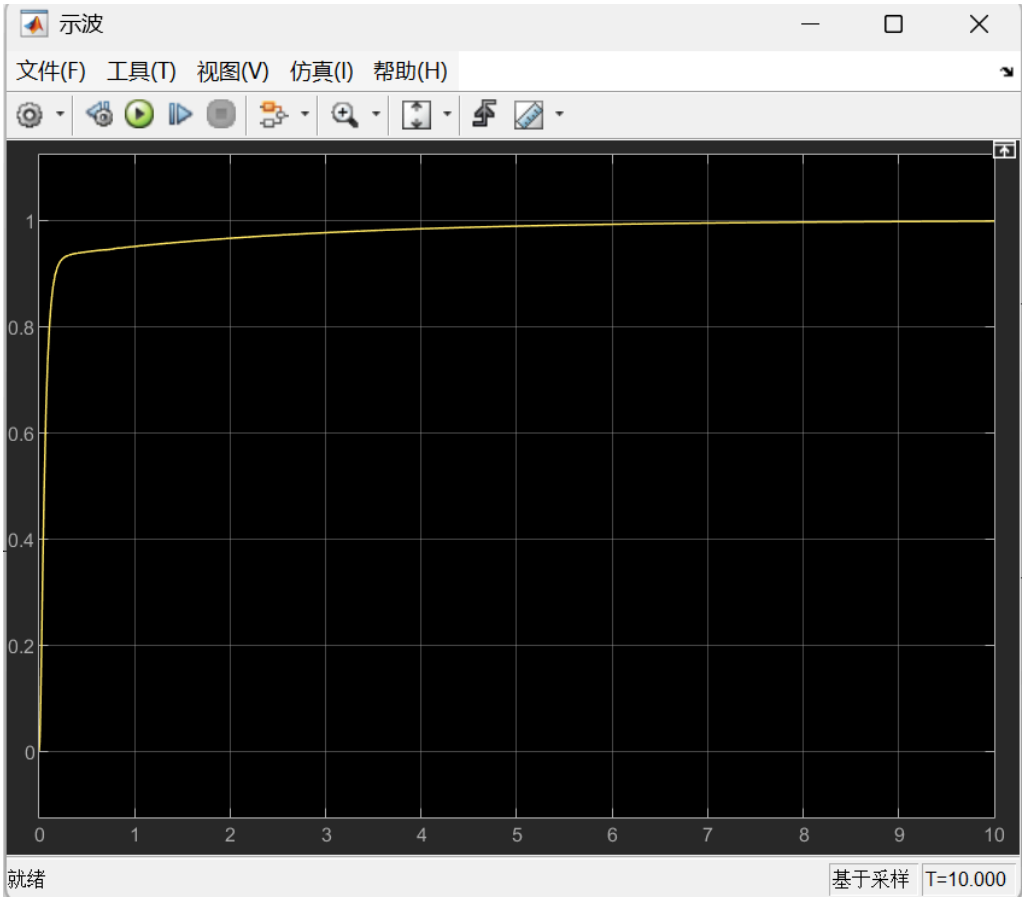

Kp=400，Ki=0，Kd=0




Kp=400，Ki=100，Kd=0



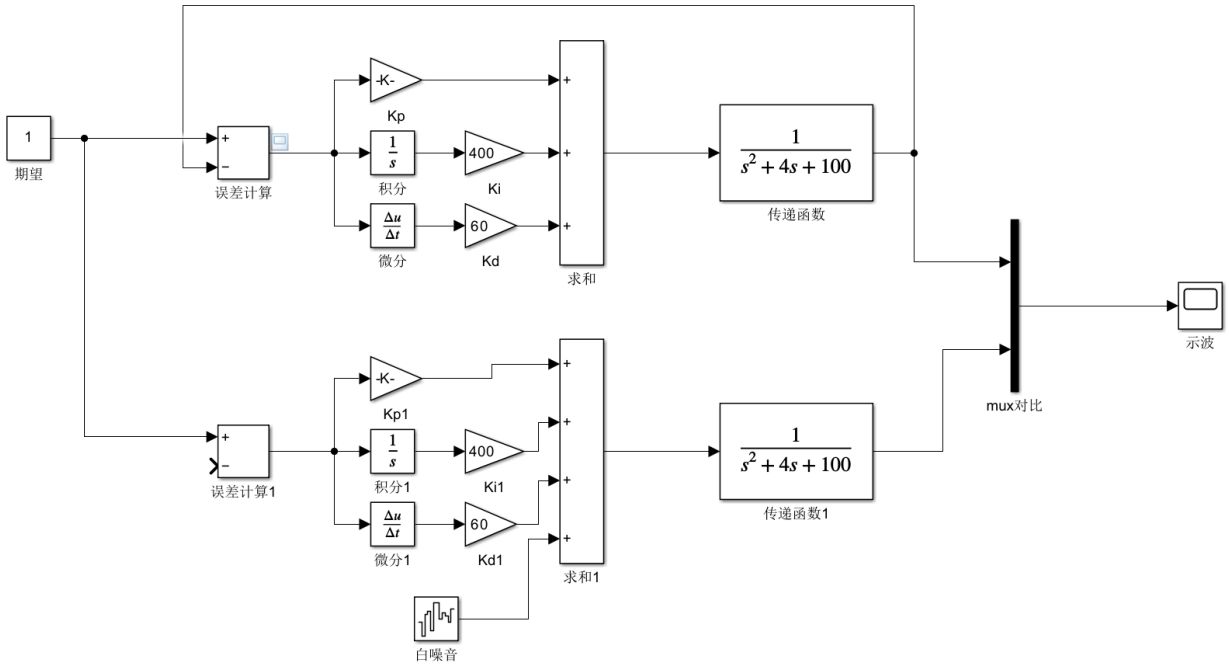

Kp=400，Ki=100，Kd=40




Kp=1000，Ki=400，Kd=60（最终结果）

- 可以看到Kd非常的强劲，直接把超调给压住了。

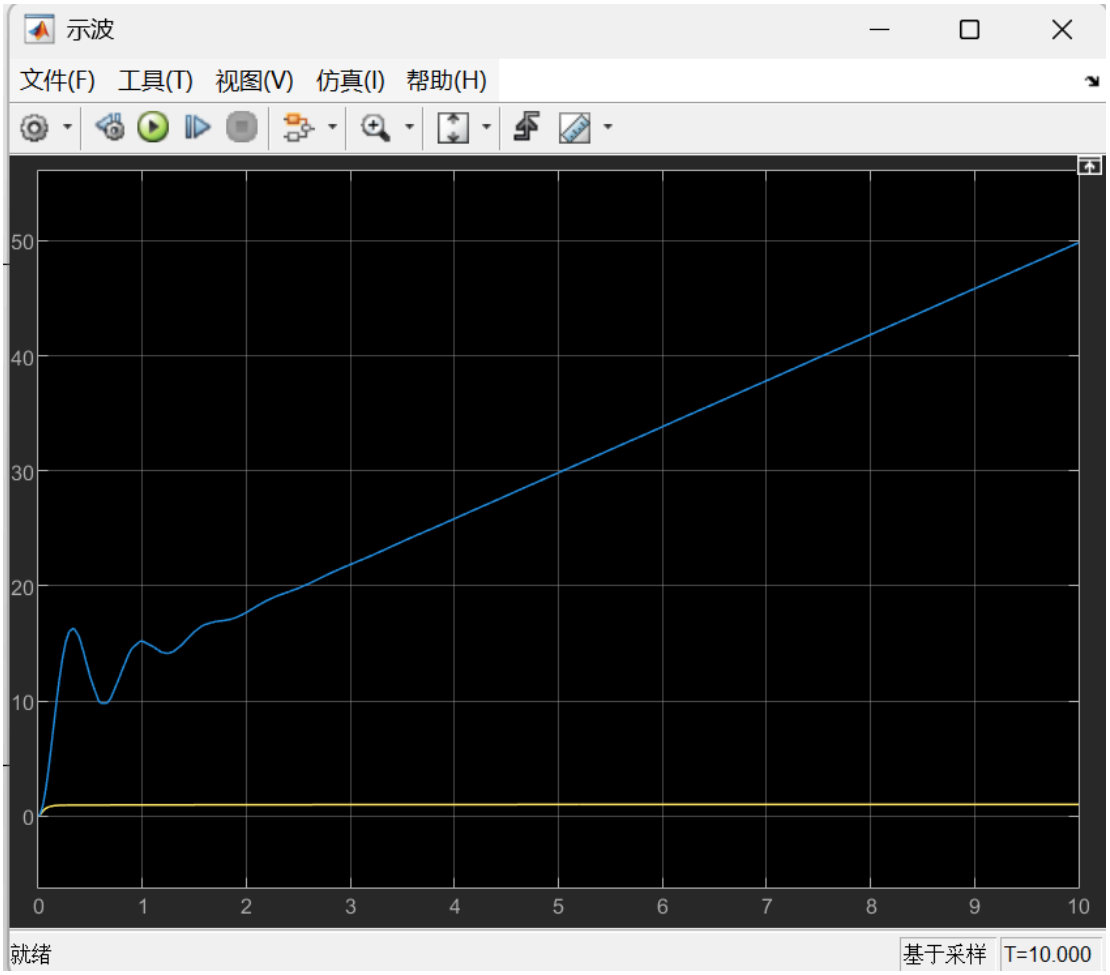
- 当然加入微分项后也有缺点，过度的kd会使系统变迟钝，导致收敛速度变慢。同时对于干扰较强的情况，也不建议加入kd。或者需要进行滤波。简单的滤波会在后面提及。
  - 我们可以用simulink，来观察引入哪怕是微小的干扰，会对系统造成什么样的影响
  - 在搜索栏中搜索 `band-limited white noise` 模块和 `mux` 模块
  - 搭建如图所示仿真（matlab在框中内容后按右键拖动可以复制）







  - 可以看到如图所示的运行结果，白噪音巧设连环计，系统误上断头台。哪怕最大功率只有0.0001的快速变化的白噪音，也可以对Kd造成灾难性打击

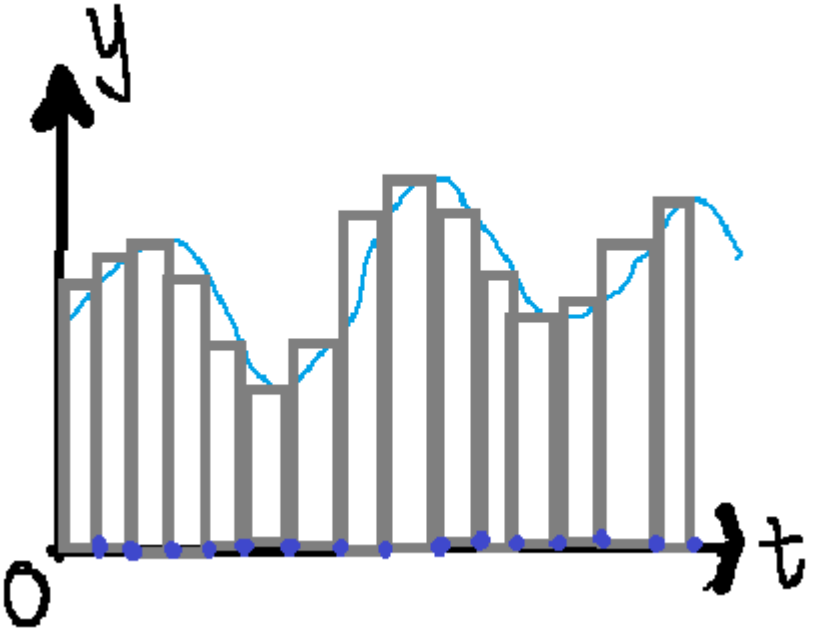



## 4.总结

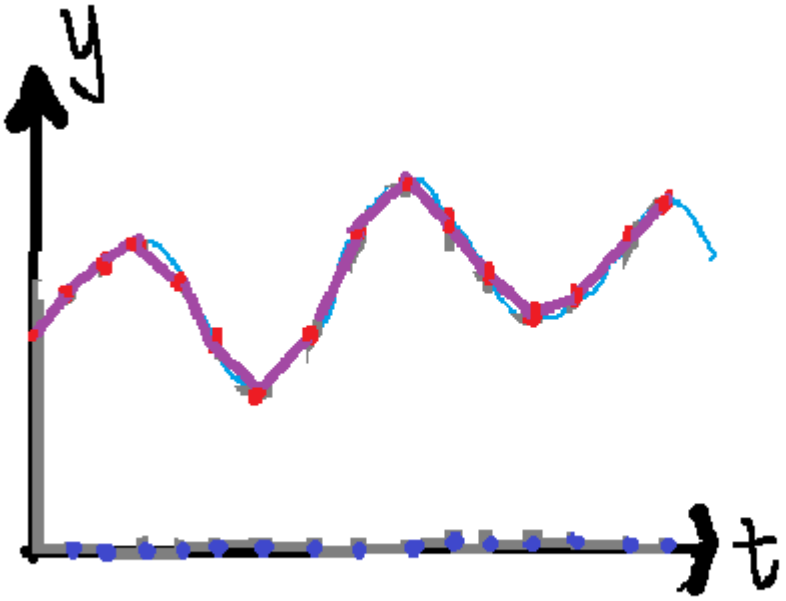
- 用文艺的说法，PID三个参数关注的分别是系统的现在，过去和未来
  - 比例控制P决定了系统响应的迅速性，能够加快调节的速度，使系统更快到达期望值，但由于稳态误差的存在，仅有P控制器的存在并不能到达期望值，而总是有偏差存在。而且，如果P控制器的参数过大，增长曲线的斜率也会较大，导致震荡幅度增大，系统震荡更加频繁，导致系统不稳定。
  - 积分控制I是为了消除系统的稳态误差，使系统到达期望值，但是这会导致调节时间变长，超调量增加。
  - 微分控制D决定了控制系统是否具有前瞻性，也就是影响着系统的未来，能够减少调节时间，使系统更快到达期望值，还能减少超调量。但是微分控制器容易受到干扰，当有外界干扰存在时，D的参数越大，系统越不稳定。
- pid最大的优点是**不需要知道系统的具体模型**。pid不关心内部具体的执行过程，更不关心系统的状态空间模型，而是根据系统的反馈量进行调节，根据反馈量不同可以设计不同的PID控制器，需要调节的参数也只有三个，使用起来简单、灵活、方便。非常适合这种黑箱式的控制。
- pid这种哪怕对系统未知也可以调参的优点，在能够得到系统的精确模型的时候会变成缺点。因为它的参数整定依赖经验配凑，最后的控制效果往往比起精细的控制算法（比如lqr）较为一般。

# 四.pid的离散化

- 我们上述的pid过程都是连续型的，但是现实中开发的嵌入式系统都是离散的系统，也就是说我们的系统有最小的时间单元，不能进一步细分。因此我们的pid公式也需要离散化的表示。
- 误差值 $e(t)$ 可以直接用期望减去当前值表示，也即 $r(n)-y(n)$
- 比例项与时间无关，因此与先前一样，有 $U(n)=K_p*e(n)$ ，注意由于离散化了，因此自变量从时间t变成了取点n
- 对于积分，我们采用矩形近似累加的方式。如下图所示，蓝线是实际的模拟信号，我们以固定的采样间隔进行采样计算控制，在t轴上有着一系列等间隔排列的紫色小点，代表我们的采样时刻,它们之间的间隔代表我们的采样周期 $T_s$ ,就像微积分一样，我们可以知道，积分等于曲线包围t轴围成的面积。
  - 用公式表示，就是 $u_i(n)=K_i*\sum_{k=0}^{n}e(k)\Delta t$
  - 当然由于采样周期 $\Delta t$ 是固定值，也可以把 $\Delta t$ 提出来与 $K_i$ 合并




- 对于微分，我们采用一阶后向差分的方式。如下图所示的方式，蓝线是实际的模拟信号，我们以固定的采样间隔进行采样计算控制，在t轴上有着一系列等间隔排列的紫色小点，代表我们的采样时刻,它们之间的间隔代表我们的采样周期 $T_s$ 。
  - 用公式表示，就是 $u(n)=K_d*\frac{(e(n)-e(n-1))}{\Delta t}$
  - 同样由于 $\Delta t$ 是固定值，也可以把 $\Delta t$ 提出来与 $K_d$ 合并




- 综上所述就得到了最后的离散化结果

$$u_i(n) = K_p * e(n) + K_i * \sum_{k=0}^{n} e(k)\Delta t + K_d * \frac{(e(n) - e(n-1))}{\Delta t}$$

# 五.pid的优化

- 到此为止，pid的大厦已经建成，但是还有几朵乌云在pid的上空需要处理，因此我们需要对基本的pid算法进行一些优化。

## 1.积分饱和和输出限幅

- 由于电机输出是有极限的，机械结构的负载也是有极限的，因此为了避免被机械抽陀螺也为了电机的生命健康，给予电机的力矩电流/电压是不能无限大的，输出限幅保证了控制信号始终在执行器能够安全运行的范围内。另一方面输出限幅也可以避免一开始的浪涌输出过大，保障系统平稳运行。
- 如下是输出限幅的基本逻辑
  - $$f(x) = \begin{cases} U_{max}, & x > x_{threhold} \\ x, & x_{threhold} > x > -x_{threhold} \\ -U_{max}, & x < -x_{threhold} \end{cases}$$
- 而基于实现输出限幅的目的，便有了配套用的积分饱和。

- 积分饱和就是当当前值与期望值相差较大的时候，积分项会快速积累扩大，但是由于执行器（比如电机）能力有限，实际输出被限幅卡在最大值无法再增加，但积分是不会停止的，导致积分积累到巨大的值，使其对应的积分控制输出远超输出最大值。当误差开始反向减小时，由于积分太大，需要很长时间才能降下来。导致系统出现巨大的超调和长时间的震荡，控制性能严重恶化。
- 积分饱和逻辑也很简单，当发生输出限幅时，不进行积分，否则进行积分即可

## 2.积分分离和变速积分

- 防止积分项过大的另一种手段是引入积分分离，当单次误差超出某个数值的时候，不计入积分项中。等待误差缩小之后再引入积分项
- 变速积分式PID是积分分离式PID的pro版本，积分分离式的权重要么是1要么是0，而变速积分PID的积分权重会动态变化，变化幅度取决于偏差大小，偏差越大，积分越慢，优缺点与积分分离式PID类似，但由于动态权重的存在，比积分分离式PID更加合理。
- 注意这种动态计入误差的方式需要经验判断误差什么时候要计入，什么时候不要，参数选择很重要。

## 3.死区

- 在实际控制中，有一点误差往往是难以消除的，进而在PID的作用下，当前值会在期望值前后高频抖动，尤其在里程环控制中非常令人恼火😤。因此有时会引入死区。当当前值与期望值的误差在死区设定值之内的时候，PID不会进行计算，只会存储当前误差值用于后续的计算，从而让系统能够稳定。
- 死区的缺点是必须忍受静态误差的存在，太小了死区没什么用，太大了静态误差会影响控制，需要自行把握

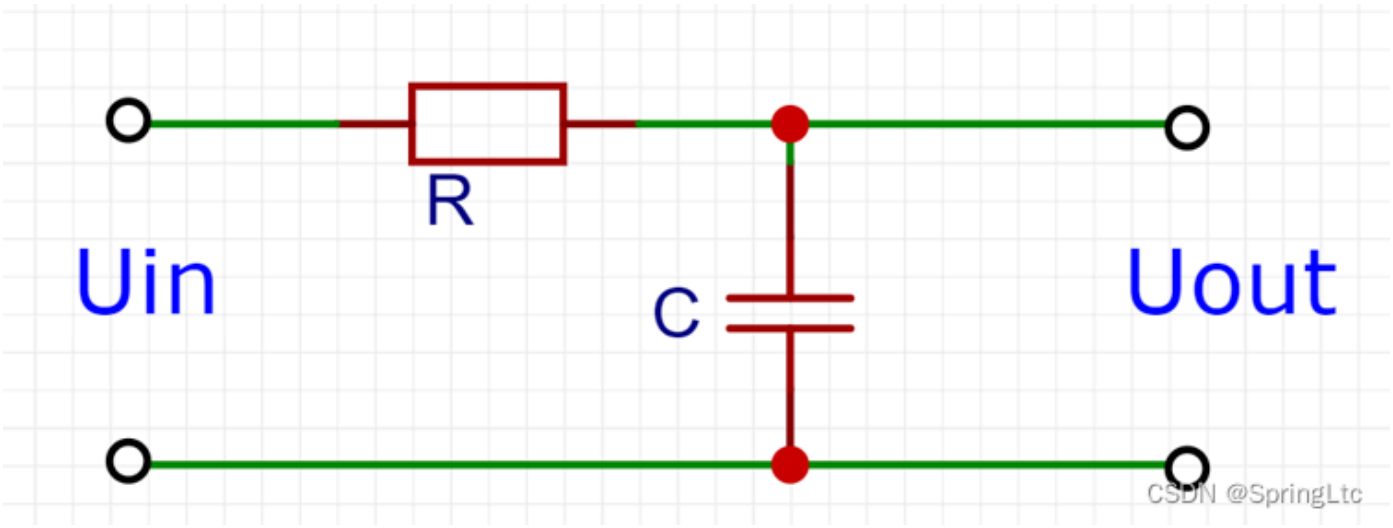
## 4.微分先行

- 由于 $K_d$ 对于微小扰动过于敏感，且当我们的期望值在运行时更改的时候，误差的突变会使得微分项突然极其大，因此一种合理的想法就是换掉误差的微分项。而微分先行就是把 $\frac{de(t)}{dt}$ 换成了 $\frac{dy(t)}{dt}$ 。也就是从误差微分变成当前值微分。这样系统误差值突变的时候，微分项可以保持平缓，并且仍然保留了对抗当前值剧烈波动的功能
- 那么。代价是什么呢？由于误差比当前值对系统的变化更敏感，使用微分先行会导致系统的响应速度下降

## 5.滤波

- 对微小扰动更直接的处理，是采用滤波算法对当前值进行滤波，除去毛刺。

- 滤波算法非常多样，我们这里介绍一种常见且实用性很高的一阶低通滤波。这里低通的意思是通低频阻高频。可以很好的去除测量当前值时出现的噪音。它的原理来自于电路中的一阶RC滤波器

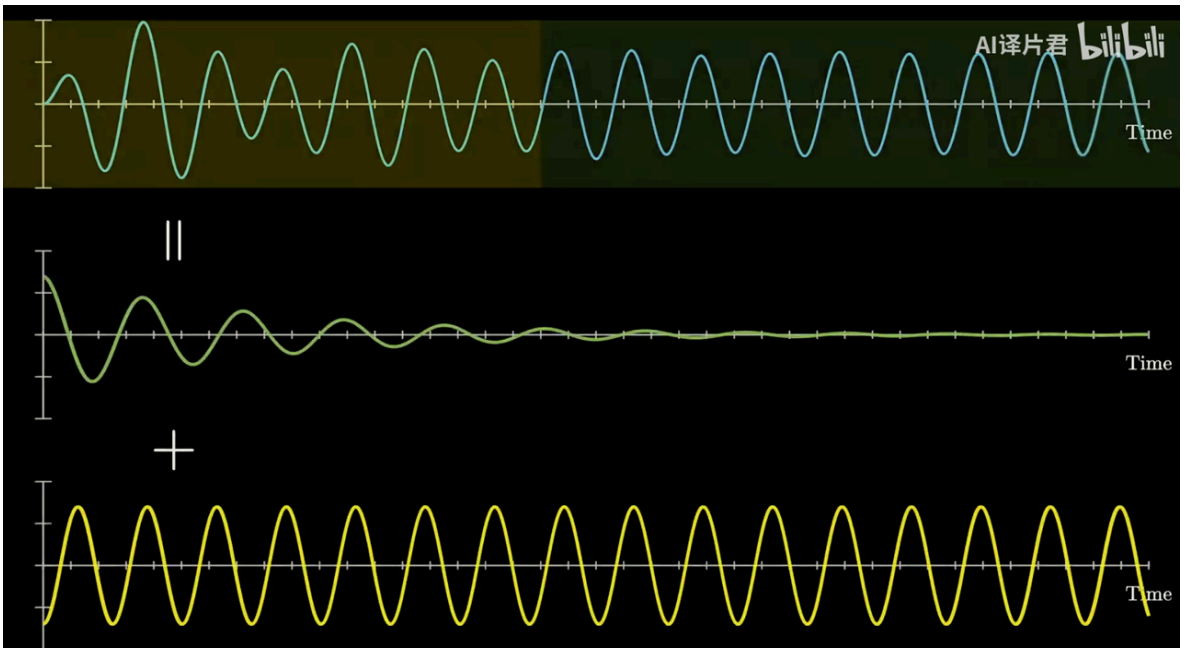



  - 在高中我们学过，电容通交流阻直流，更进一步地我们有容抗公式 $X_C = \frac{1}{2\pi f C}$ ，单位是欧姆。因此我们可以把电容和电阻联系起来。在高频电流通过时， $f$ 很大，因而容抗很小，而 $U_{out}$ 负载处的电阻大，因而电容分得电流更多，高频电流被直接泄放到地，从而起到了滤去高频电流的作用。

- 基于上面粗略的原理解释，我们就可以定性地剖析滤波的公式了。高中学过电容有 $C = \frac{Q}{U}$ ，同时电荷量 $Q$ 是电流 $i(t)$ 在时间上的积分，因此移项求导后有 $i_C = C\frac{dU_C}{dt}$ ,这就是电容分压与电流之间的关系式。
- 由于负载与电容并联，电压相等，为简化计算，我们姑且认为负载阻值很大，近似开路，只计算电容分压。电容分压公式有 $u_C = Ri_C = RC\frac{dU_{out}}{dt}$ ,因此电路整体分压有 $U_{in}(t) = RC\frac{dU_{out}(t)}{dt} + U_{out}(t)$ 。为了实际应用，我们往往把方程离散化，并用差分替代导数，那么就有 $RC\frac{U_{out}(n) - U_{out}(n-1)}{T_s} + U_{out}(n) = U_{in}(n)$ ，化简之后把 $U_{out}$ 项合并，可以得到 $U_{out}(n) = \frac{T_s}{RC + T_s}U_{in}(n) + \frac{RC}{RC + T_s}U_{out}(n-1)$ 。因此我们令**平滑因子** $\alpha = \frac{T_s}{RC + T_s}$ ,就可以得到 $U_{out}(n) = \alpha U_{in}(n) + (1-\alpha)U_{out}(n-1)$ ，由此我们就可以推导出一阶低通滤波的公式。当 $\alpha$ 越大，滤波器相信当前数据越多，响应更快，平滑效果差；越小，相信历史数据越多，平滑效果越好，但响应迟缓，相位滞后大。
- 一阶低通滤波的本质，是把当前时刻的输入（也就是拿到的原始数据）和上一时刻的输出（上次滤波完的数据）进行加权平均

## 5(补) 截止频率

- 事实上对于 $U_{in}(t)=RC\frac{dU_{out}(t)}{dt}+U_{out}(t)$，对前面的数学基础有印象的会知道这就是一阶惯性系统。我们同样可以求出输入和输出电压之间的传递函数 $H(s)=\frac{1}{RC*s+1}$，我们设输入 $x(t)=e^{i\omega t}$ (由欧拉公式，指数函数可以分解成三角函数，所以本质是sin的三角函数输入也可以看成 $e^{i\omega t}$ 与 $e^{-i\omega t}$ 的叠加。为了方便这里就只算一半），那么 $X(s)=\frac{1}{s-i\omega}$，$Y(s)=H(s)X(s)$。由式子我们知道输出存在两个极点，系统极点 $s=-\frac{1}{RC}<0$，表明系统本身运动趋于收敛，输入的极点 $s=i\omega$，表明输入恒定震动，两者叠加就是系统最后趋势。显然两者结合后最后趋于和输入频率相同的震动




供辅助理解什么是系统本身运动和输入的叠加

- 对这两个极点，我们一样使用部分分式法，有 $Y(s)=\frac{K_1}{s+\frac{1}{RC}}+\frac{K_2}{s-i\omega}$，同乘 $s-i\omega$，再令 $s=i\omega$，我们就有 $K_2=\frac{1}{RC\cdot i\omega+1}=H(i\omega)$。当时间趋于无穷的时候，$K_1$ 项由于衰减会趋于0，所以我们考虑稳定态可以只考虑 $K_2$。做拉普拉斯逆变换，$y(t)=\mathcal{L}^{-1}\left(\frac{K_2}{s-i\omega}\right)=K_2e^{i\omega t}=H(i\omega)e^{i\omega t}$,所以我们得到了关键公式 $y(t)=H(i\omega)*x(t)$，这就是趋于稳定态的时候，时域上输入和输出电压的变化。
- 我们高中就知道，复数包含幅角和模长两部分，在 $H(i\omega)$ 中，模长就意味着滤波前后的衰减，幅角就意味着滤波前后波的偏移。而 $\omega$ 就是输入和输出电压的频率，由此我们就可以引入截止频率了。
- 我们知道 $\frac{1}{RC\cdot i\omega+1}=H(i\omega)$，令 $\omega_c=\frac{1}{RC}$，就是我们常说的截止频率。代入，有 $H(i\omega)=\frac{1}{1+i\frac{\omega}{\omega_c}}$,那么幅值和幅角就有

- $|H(i\omega)| = \dfrac{1}{\sqrt{1+\left(\frac{\omega}{\omega_c}\right)^2}}$
- $\theta(\omega) = -\arctan\left(\dfrac{\omega}{\omega_c}\right)$

- 由此我们就知道截止频率为什么叫截止频率了——
  - 当输入信号的频率小于截止频率时，幅值基本等于1，也就是输入信号能基本还原出有用信号的状态
  - 当输入信号的频率大于截止频率时，幅值就迅速小于1了，也就达到了衰减高频干扰信号的目的
  - 当输入信号的频率等于截止频率时，其输出幅值衰减为输入的约0.707倍
- 还记得平滑因子 $\alpha = \dfrac{T_s}{RC+T_s}$ 吗，当 $T_s << RC$ 的时候，就有 $\alpha = T_s * \omega_c$ ，也就得到了小量近似后，截止频率和平滑因子内含的关系。
  - 这个公式对数字滤波器具有实际意义，因为mcu的计算必须是离散的，而mcu极快的采样速度又能满足 $T_s << RC$ ，由此可以推出大概合理的平滑因子区间

# 六.补充

- 这里提供两个有趣的小项目，可以帮助大家对pid有更直观的了解
- 这是一个在excel中实现pid及其拓展的文件，这样大家不需要先写代码也可以对pid的控制有大概的认知，同时也规避了传递函数这个相对复杂的概念

| 系统参数 | 质量m(kg) | 重力G(N) | 空气阻力系数k(N/(m/s)) | 更新周期t(s) |
|---|---|---|---|---|
| | 1 | -10 | -1 | 0.1 |
| 初始参数 | 速度v0(m/s) | 位置x0(m) | | |
| | 0 | 0 | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

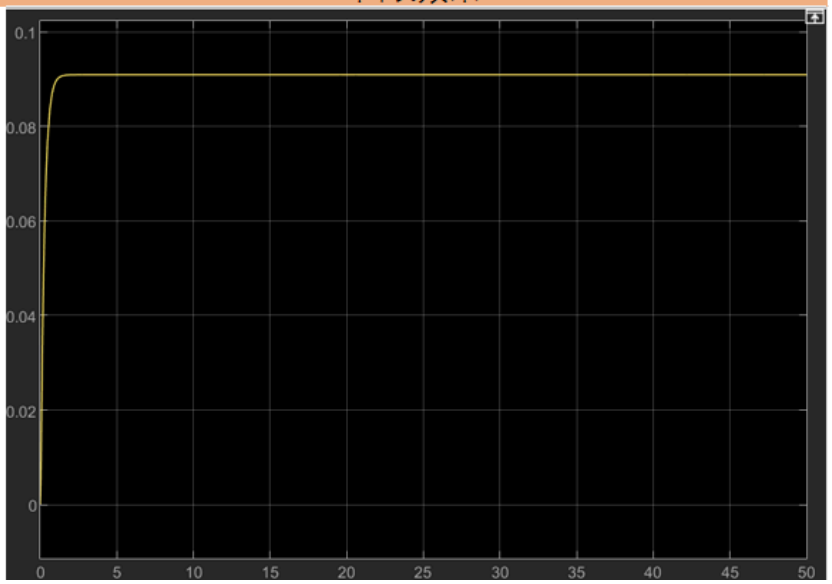
| 更新次数 | 时间t(s) | 物体受力F(N) | 物体加速度a(m/s^2) | 物体速度v(m/s) |
|---|---|---|---|---|
| 0.000 | 0.000 | 0.000 | -10.000 | 0.000 |
| 1.000 | 0.100 | 0.000 | -10.000 | -1.000 |
| 2.000 | 0.200 | 0.000 | -9.000 | -1.900 |
| 3.000 | 0.300 | 0.000 | -8.100 | -2.710 |
| 4.000 | 0.400 | 0.000 | -7.290 | -3.439 |
| 5.000 | 0.500 | 0.000 | -6.561 | -4.095 |
| 6.000 | 0.600 | 0.000 | -5.905 | -4.686 |
| 7.000 | 0.700 | 0.000 | -5.314 | -5.217 |
| 8.000 | 0.800 | 0.000 | -4.783 | -5.695 |
| 9.000 | 0.900 | 0.000 | -4.305 | -6.126 |
| 10.000 | 1.000 | 0.000 | -3.874 | -6.513 |
| 11.000 | 1.100 | 0.000 | -3.487 | -6.862 |
| 12.000 | 1.200 | 0.000 | -3.138 | -7.176 |
| 13.000 | 1.300 | 0.000 | -2.824 | -7.458 |
| 14.000 | 1.400 | 0.000 | -2.542 | -7.712 |
| 15.000 | 1.500 | 0.000 | -2.288 | -7.941 |
| 16.000 | 1.600 | 0.000 | -2.059 | -8.147 |
| 17.000 | 1.700 | 0.000 | -1.853 | -8.332 |
| 18.000 | 1.800 | 0.000 | -1.668 | -8.499 |
| 19.000 | 1.900 | 0.000 | -1.501 | -8.649 |
| 20.000 | 2.000 | 0.000 | -1.351 | -8.784 |
| 21.000 | 2.100 | 0.000 | -1.216 | -8.906 |
| 22.000 | 2.200 | 0.000 | -1.094 | -9.015 |
| 23.000 | 2.300 | 0.000 | -0.985 | -9.114 |
| 24.000 | 2.400 | 0.000 | -0.886 | -9.202 |
| 25.000 | 2.500 | 0.000 | -0.798 | -9.282 |

pid.xlsx

- https://www.bilibili.com/video/BV1BL4y1J79y这个视频用matlab的simscape和simulink实现了控制领域一个非常经典的模型——一阶倒立摆，同样也可以帮助大家对pid有更直观的认知。

## 七.任务

1. 请用simulink对传递函数 $\frac{1}{s^2+20s+100}$ 和 $\frac{1}{s^4+3s^3+3s^2+s}$ 的PID参数进行调节，并将调试过程中每一关键步设置的PID参数和对应的系统响应图片整理到excel表格中。表格要求如下所示（允许将仿真停止时间设置到50秒）

| Kp | Ki | Kd | 图表效果 |
|---|---|---|---|
| 10 | 0 | 0 |  |



2. 请大家根据pid的算法原理，写出自己的pid库，要求在pid的优化这一章节中任选两个进行体现，并用注释进行标明。注意.h文件的代码规范，把函数的声明放在.h里面。提交时请提交独立文件夹，包含你的.c/.cpp和.h文件

代码块

```c
//如下是pid结构体（类）的演示和基本处理过程的提示，仅供参考，欢迎大家提供更好的写法
//对于C语言版本
typedef struct
{
        float reference;
        float feedback;
        float delta_t;
        float error;
        float prev_error;
        float Kp;
        float Ki;
        float Kd;
        float Pout;
        float Iout;
        float Dout;
        float Output;
}
PID_t;

void PID_Init(PID_t* pid,float Kp, float Ki, float Kd,float delta_t)
{
```

```
        pid->reference=0;
        pid->feedback=0;
        pid->delta_t = delta_t;
        pid->error=0;
        pid->prev_error=0;
        pid->Kp=Kp;
        pid->Ki=Ki;
        pid->Kd=Kd;
        pid->Pout=0;
        pid->Iout=0;
        pid->Dout=0;
        pid->Output=0;
}

float PID_Calc(float reference, float feedback)
{
        //计算本次误差

        //计算Pout

        //计算Iout

        //计算Dout

        //得到总输出

        //更新数据

        //返回输出结果
}

//对于C++版本
class class_pid
{
public:
void init(float _Kp,float _Ki,float _Kd,float _delta_t);
void calc();
float get_out();

void set_kp(float _kp);
void set_ki(float _ki);
void set_kd(float _kd);
void set_dt(float _delta_t);
void set_reference(float _reference);
void set_feedback(float _feedback);
void set_Iout(float _Iout);
void set_Output(float _Output);
```

```
protected:
float reference = 0.0f;
float feedback = 0.0f;
float delta_t = 0.1f;
float error = 0.0f;
float prev_error = 0.0f;
float Kp;
float Ki;
float Kd;
float Pout;
float Iout;
float Dout;
float Output;
};

void class_pid::init(float _Kp,float _Ki,float _Kd,float _delta_t)
{
Kp = _Kp;
Ki = _Ki;
Kd = _Kd;
delta_t = _delta_t;
}
void class_pid::calc()
{
        //计算本次误差

        //计算Pout

        //计算Iout

        //计算Dout

        //得到总输出

        //更新数据

        //返回输出结果
}
//……(此处省略各种set函数的执行代码)
```

3. 在pwm的任务中，我们已经学过了如何控制电机转动并记录其速度。现在请将pid算法引入电机的控制中，尝试通过pid对电机进行速度环闭环控制，并使用任意示波软件(vofa+,jcom,serial wire……）进行示波展示。要求展示速度设定值，速度测量值和输出占空比。视频要求：提交两个视频，第一个视频用手转动电机，示波从编码器读取到的速度。第二个视频要求展示输入期望速度后，电机转速在pid控制下达到期望速度的过程

# 八.回顾

1. 如何建立自动控制系统？思考在不同的控制系统中，什么是受控量，什么是参考量，什么是输出量，在实际编程时如何映射。
2. PID算法的原理是什么？三种控制各自的作用的及特点是什么？
3. 如何离散化PID算法？
4. 如何实现PID算法编程？
5. 原始版本的PID算法有什么不足？

——**by lxt**

qq:350208451